claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-a556f3c566b6adefa.jsonl`
- SHA-256: `27122a04206a67f71214a614fcc9ce531bac0d7e1f590f5244502d6e8079b39f`
- Source modified: `2026-06-21T14:01:51+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:52:09.112Z)

Research bundling/obtaining MODEL WEIGHTS and FFMPEG for a downloadable app. Weights: download-on-first-run from Hugging Face Hub vs a public GCS/CDN bucket vs vendoring in the wheel (PyPI size limits ~100MB), checksum verification, offline/cached, licensing of the weights. FFMPEG: imageio-ffmpeg (bundled static binary, license?), static ffmpeg builds (BtbN / gyan.dev, GPL vs LGPL implications for redistribution!), system-install instructions, shutil.which discovery + a configurable path. Recommend a first-run setup that "just works" on Windows for a non-tech user. Use web search.

Return the RESEARCH schema. Context about our app:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi).

The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).
khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

2. assistant (2026-06-21T13:52:12.304Z)

I'll research this thoroughly. Let me start by reading the actual files in the repo to ground my recommendations, then do web research on the packaging concerns.

3. user (2026-06-21T13:52:13.266Z)

```
1       # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2       #
3       # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4       # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5       # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6       # auto-detection would otherwise miss the un-__init__'d `backend` root.
7       #
8       # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9       # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10      # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11      # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12      [build-system]
13      requires = ["hatchling"]
14      build-backend = "hatchling.build"
15
16      [project]
17      name = "badminton-highlight-indexer"
18      version = "0.1.0"
19      description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20      readme = "README.md"
21      requires-python = ">=3.10"
22      license = { text = "MIT" }
23      authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24      keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26      # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27      dependencies = [
28          "fastapi>=0.111.0",
29          "uvicorn>=0.30.1",
30          "opencv-python>=4.9.0.80",
31          "numpy>=1.26.4",
32          "pydantic>=2.7.1",
33          "jinja2>=3.1.4",
34          "python-dotenv>=1.0.0",
35          "google-genai>=2.4.0",
36          "supervision>=0.21.0,<0.30.0",
37      ]
38
39      [project.optional-dependencies]
40      # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41      dev = [
42          "pytest",
43          "pytest-cov",
44          "httpx",        # fastapi TestClient transport
45          "ruff",
46          "mypy",
47      ]
48
```

```
49      # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50      # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51      # fetched over SSH; if you lack access, simply omit this group ? reporting
52      # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53      #   pip install -e ../sports-obsreport --no-build-isolation
54      reporting = [
55          "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56      ]
57
58      # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59      # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60      # express, so they are NOT declarable here. Install them out-of-band instead:
61      #   # CUDA 12.4 box:
62      #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63      #   # CPU-only box (also what CI uses):
64      #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65      gpu = []
66
67      # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68      # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69      # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70      # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
71      # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
72      storage-s3 = ["boto3>=1.34"]
73      storage-gcs = ["google-cloud-storage>=2.16"]
74
75      [project.scripts]
76      # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77      indexer = "backend.main:main"
78
79      [tool.hatch.metadata]
80      # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81      # reference*); hatchling requires opting in to allow those in metadata.
82      allow-direct-references = true
83
84      [tool.hatch.build.targets.wheel]
85      # `backend` is a namespace package (no __init__.py); list both import roots
86      # explicitly so an editable/wheel install exposes `import backend` and
87      # `import training` regardless of cwd.
88      packages = ["backend", "training"]
89
```

4. user (2026-06-21T13:52:14.041Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
```

```
        court_green_lower: list[int] = Field(default=[35, 40, 40])
        court_green_upper: list[int] = Field(default=[85, 255, 255])
        court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


    class ValidationConfig(BaseModel):
        min_resolution_width: int = 1280
        min_resolution_height: int = 720
        min_fps: float = 29.9
        min_blur_threshold: float = 20.0
        max_scene_cuts_per_minute: float = 0.5
        relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)


    class TrackNetConfig(BaseModel):
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""
        queue_length: int = 5
        stage_in_wsl: bool = True
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
        # decoded frame cache) are deleted automatically ? they are pure intermediates,
        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
        # 0 = disabled.
        min_free_gb: float = 0.0


    class WasbIndexConfig(BaseModel):
        """Typed config for the live WASB shuttle substrate (indexing.wasb).

        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
        these knobs actually take effect ? previously this whole block was silently dropped
        because nested config sections defaulted to extra='ignore'.
        """
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
        # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
        # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
        # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
        device: str = "cuda"        # auto | cuda | mps | cpu
        python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
        # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
        # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
```

```
regenerable
78          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79          keep_frames: bool = False
80          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81          min_free_gb: float = 0.0
82          # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83          # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84          # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85          # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86          # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87          # Tri-state:
88          #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89          #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90          #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91          stream_video: Optional[bool] = None
92
93
94      class FusionConfig(BaseModel):
95          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97          Every default reproduces control behaviour: the fusion segmenter persists the
98          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99          true ? the person-cue features, but it does NOT gate the served handoff unless a
100         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102         supplied ? off-court persons (spectators / adjacent courts) are excluded.
103         """
104         # Which trajectory substrate seeds the windows (shuttle stays primary).
105         substrate: Literal["wasb", "tracknet"] = "wasb"
106         # Windowing velocity threshold for the fusion family (the engine reads
107         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109         velocity_threshold: float = 0.02
110         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111         # enabled ? so the default path never imports torch / touches the GPU.
112         cue_flags: dict[str, bool] = Field(default_factory=dict)
113         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115         min_crossings: int = Field(default=0, ge=0)
116         # Served Gate B: when the person cue is on, drop windows with median in-court
players
117         # < this. 0 = OFF.
118         min_players: int = Field(default=0, ge=0)
119         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120         court_polygon: Optional[list[list[float]]] = None
121         # Net line for the person two-sided-motion split (person features only ? the shuttle
122         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123         net_axis: Literal["x", "y"] = "y"
124         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
```

```
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135         detector_impl: str = "auto"
136         motion_threshold: float = 0.08
137         min_segment_duration: float = 3.0
138         dead_time_merge_gap: float = 2.0
139         chunk_duration: float = 90.0
140         chunk_overlap: float = 10.0
141         detector_model: str = "fasterrcnn_resnet50_fpn"
142         detector_score_threshold: float = 0.5
143         yolo_velocity_threshold: float = 0.15
144         yolo_frame_skip: int = 3
145         yolo_tracker: str = "bytetrack.yaml"
146         yolo_prefetch_workers: int = 2
147         min_action_window_duration: float = 2.0
148         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
149         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
151         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155         max_window_duration: float = 6.0
156         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
157         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160         # Only cuts (recall can't drop). OFF by default until measured/promoted.
161         window_hard_cap: bool = False
162         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169         window_inactivity_end: float = 1.5
170         inactivity_rally_thresh: float = 0.20
171         inactivity_min_density: float = 0.30
```

```
172         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180         window_blob_fallback: bool = False
181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
190         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191         # chunks of a high-bitrate (4K) source blow past the provider upload cap
192         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195         # Matches gemini_refine's --clip-scale-width default.
196         ai_chunk_scale_width: int = 1280
197         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202         skip_ai_handoff: bool = False
203         # Optional debug knob: trim every video to the first N seconds before processing.
204         debug_duration: Optional[float] = None
205
206         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208         fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210         @field_validator("default_segmenter")
211         @classmethod
212         def segmenter_must_be_registered(cls, v: str) -> str:
213             # Registry check happens at runtime in main/segmenter selection.
214             # Here we just allow any string for forward compatibility.
215             return v
216
217         @model_validator(mode="after")
218         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220             # step makes `while t < duration: t += step` loop forever, growing memory before
```

```
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
        # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
        # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
        # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
        # local detector over-fires and its false positives are mostly SHORT (motion blips,
        # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
        # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
        min_rally_duration: float = Field(default=0.0, ge=0.0)
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
```

```
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296
297     class StorageConfig(BaseModel):
298         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
299         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
300         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
301         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
302         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
303         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
304         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
305         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
306         output_root: str = ""
307         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
308         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
309         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
310         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
311         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
312         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
313         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
314         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
315         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
316         input_root: str = ""
317         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
318         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
319         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
320         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
321         # the legacy flat layout is used unchanged.
322         single_folder_per_video: bool = False
323         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
324         workspace_root: str = ""
325         # Cloud namespace prefix under the root so per-video folders sit tidily beside
```

```
326         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
327         workspace_prefix: str = "workspaces"
328         local: dict[str, Any] = Field(default_factory=dict)
329         s3: dict[str, Any] = Field(default_factory=dict)
330         gcs: dict[str, Any] = Field(default_factory=dict)
331         gdrive: dict[str, Any] = Field(default_factory=dict)
332
333
334     class DeploymentConfig(BaseModel):
335         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
336         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
337
338         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
339         behaves exactly as before this section existed. A profile is opt-in (``config.json``
340         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
341         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
342         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
343         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
344
345         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
346
347         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
348         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
349         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
350         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
351         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
352
353
354     class BillingConfig(BaseModel):
355         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
356
357         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
358         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
359         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
360         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
361         inserts a real, calibrated version and points ``pricebook_version`` at it.
362
363         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
364         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
365         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
366
367         enabled: bool = False
368         pricebook_version: str = "placeholder-v0"   # the seeded $0 version; owner flips to
a real one
369         fx_inr_per_usd: float = 83.0                  # USD?INR display rate (Bangalore
market)
370         margin: float = 0.0                            # resale markup (?0); 0 = bill at
cost
371
372
```

```
    class AppConfig(BaseModel):
        """Root configuration object.

        All runtime configuration should be accessed via an instance of this class
        (or the loader) rather than raw dict lookups.
        """

        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
        # the alias never deprecates underneath us. Eval reports record the run date
alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
        security: SecurityConfig = Field(default_factory=SecurityConfig)
        logging: LoggingConfig = Field(default_factory=LoggingConfig)
        storage: StorageConfig = Field(default_factory=StorageConfig)
        deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
        billing: BillingConfig = Field(default_factory=BillingConfig)

        # Allow extra keys during transition so existing config.json files do not break.
        model_config = {
            "extra": "allow",
            "validate_assignment": True,
        }

        # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
        # Will be removed once all call sites (validators, segmenters, etc.) move to typed
        # attribute access / the typed config. A nested *section* is returned as a plain
dict
        # via model_dump, so legacy chained access keeps working ? returning the Pydantic
        # section model instead breaks it (a model has no .get), which bit every validator
on
        # real runs. Scalar leaves are returned as-is.
        def get(self, key: str, default: Any = None) -> Any:
            _MISSING = object()
            val: Any = _MISSING
            if key in type(self).model_fields or hasattr(self, key):
                val = getattr(self, key, _MISSING)
            if val is _MISSING:
                # Legacy convenience: find a leaf key inside any nested section. Sections
are
                # discovered dynamically from the model fields, so a future top-level
section
                # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
                for fname in type(self).model_fields:
                    section = getattr(self, fname, None)
                    if isinstance(section, BaseModel) and hasattr(section, key):
                        val = getattr(section, key)
                        break
            if val is _MISSING:
                return default
            if isinstance(val, BaseModel):
                return val.model_dump(mode="json")
            return val

        def __getitem__(self, key: str) -> Any:
            val = self.get(key)
```

```
430             if val is None and key not in type(self).model_fields:
431                 # Re-raise like a real dict for unknown top-level
432                 raise KeyError(key)
433             return val
434
435
436     # Convenience type alias for callers that want to annotate
437     Config = AppConfig
438
```

5. assistant (2026-06-21T13:52:17.197Z)

I have the config models. Now let me read the storage layer (especially GCS weights handling), the WASB runner, and any existing packaging/desktop docs to ground the research.

6. user (2026-06-21T13:52:17.853Z)

```
backend\storage\gcs.py
backend\storage\local.py
backend\storage\s3.py
backend\storage\base.py
backend\storage\gdrive.py
backend\storage\ref.py
backend\storage\__init__.py
```

7. user (2026-06-21T13:52:17.982Z)

```
No files found
```

8. assistant (2026-06-21T13:52:20.513Z)

No dedicated ffmpeg module. Let me find how ffmpeg/ffprobe are invoked and how weights are downloaded.

9. user (2026-06-21T13:52:21.614Z)

```
Found 32 files
backend\main.py
tests\test_api_endpoints.py
tests\test_compute_backend.py
backend\worker\__main__.py
tests\test_profile_parity.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
tests\test_worker.py
backend\pipeline\detectors\wasb_infer.py
scripts\doctor.py
backend\pipeline\stitcher\compiler.py
tools\generate_highlights.py
backend\eval\fusion_audio.py
tests\test_audio_features.py
tests\test_compiler.py
tests\test_validators_base.py
backend\utils\run_telemetry.py
tests\test_watermark.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\validators\base.py
backend\utils\video.py
tests\test_video.py
tests\test_reporting.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\format.py
backend\reporting\harvest.py
```

```
tests\test_reporting_harvest.py
tests\test_rally_slicer.py
tests\test_hit_detector.py
backend\pipeline\audio\__init__.py
backend\tools\rally_slicer.py
```

10. user (2026-06-21T13:52:22.547Z)

```
tools\cleanup_caches.py:130:    if not shutil.which("wsl"):
scripts\doctor.py:13:    return shutil.which(cmd) is not None
backend\utils\run_telemetry.py:62:    exe = shutil.which("nvidia-smi")
backend\utils\run_telemetry.py:109:    exe = shutil.which("nvidia-smi")
```

11. assistant (2026-06-21T13:52:26.600Z)

Let me look at how ffmpeg is actually invoked (the central video util) and the doctor script which already does dependency checks.

12. user (2026-06-21T13:52:27.556Z)

```
1       import subprocess
2       import logging
3       from typing import Optional
4
5       logger = logging.getLogger(__name__)
6
7       def get_video_duration(video_path: str) -> float:
8           """Returns the exact duration of a video file in seconds using ffprobe.
9           Returns 0.0 on failure."""
10          try:
11              duration_output = subprocess.check_output([
12                  "ffprobe", "-v", "error", "-show_entries", "format=duration",
13                  "-of", "default=noprint_wrappers=1:nokey=1", video_path
14              ]).decode().strip()
15              return float(duration_output)
16          except Exception as e:
17              logger.error(f"Could not detect video duration for {video_path}: {e}")
18              return 0.0
19
20      def extract_video_chunk(video_path: str, start_time: float, duration: float,
21                              output_path: str, reencode: bool = False,
22                              scale_width: Optional[int] = None) -> bool:
23          """Extracts a chunk from a video file.
24
25          Args:
26              reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                        If False, uses stream-copy (fast but cuts at nearest keyframe).
28                        Use True for short clips where keyframe drift matters.
29              scale_width: If set (requires reencode), downscale to this width (height auto,
30                        even). Useful for high-res sources headed to an API that analyses at
31                        low res anyway ? shrinks uploads with no analysis loss.
32          """
33          cmd = [
34              "ffmpeg", "-y",
35              "-ss", str(start_time),
36              "-t", str(duration),
37              "-i", video_path,
38          ]
39          if reencode:
40              if scale_width:
41                  cmd += ["-vf", f"scale={int(scale_width)}:-2"]
42              cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
43          else:
44              cmd += ["-c", "copy"]
```

```
45          cmd.append(output_path)
46          try:
47              subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
48              return True
49          except Exception as e:
50              logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
51              return False
52
```

13. user (2026-06-21T13:52:27.698Z)

```
1       import sys
2       import os
3       import subprocess
4       import shutil
5
6       # This diagnostics/bootstrap script lives in scripts/ but operates on the repo
7       # root (requirements.txt, config.json). Resolve the root from this file's
8       # location so it works regardless of the caller's cwd.
9       REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10
11      def check_command(cmd):
12          """Check if a system command is available."""
13          return shutil.which(cmd) is not None
14
15      def run_pip_install():
16          """Install required Python packages."""
17          print("Installing Python packages from requirements.txt...")
18
19          # Check if NVIDIA GPU is present
20          has_gpu = check_command("nvidia-smi")
21
22          requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
23          pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
requirements_path]
24
25          import platform
26          if has_gpu:
27              print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
28              pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
29          elif platform.system() == "Darwin":
30              # macOS has no CUDA; the default PyPI wheel already ships CPU + Apple MPS.
31              print("[INFO] macOS detected. Using default PyTorch wheels (CPU + Apple MPS).")
32          else:
33              print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
34              pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
35
36          try:
37              subprocess.check_call(pip_cmd)
38              print("[SUCCESS] Python packages installed.")
39              return True
40          except subprocess.CalledProcessError as e:
41              print(f"[ERROR] Failed to install Python packages: {e}")
42              return False
43
44      def init_default_config():
45          """Initialize a default config.json if it doesn't exist.
46
47          Now delegates to the canonical backend.config loader so that defaults
48          stay in sync with the Pydantic models (single source of truth).
49          """
50          try:
```

```
            from backend.config import save_default_config
        except Exception as e:
            print(f"[ERROR] Could not import backend.config loader: {e}")
            print("[INFO] Falling back to no-op for config initialization.")
            return

        config_path = os.path.join(REPO_ROOT, "config.json")
        if os.path.exists(config_path):
            print(f"[INFO] {config_path} already exists. Skipping initialization.")
            return

        try:
            saved_path = save_default_config(config_path)
            print(f"[SUCCESS] config.json initialized with default settings at
{saved_path}.")
        except Exception as e:
            print(f"[ERROR] Failed to create config.json via loader: {e}")

    def run_diagnostics():
        """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
        print("=" * 50)
        print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
        print("=" * 50)

        # 1. Check Python Version
        print(f"Python Version:
{sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
        if sys.version_info.major < 3 or (sys.version_info.major == 3 and
sys.version_info.minor < 8):
            print("[WARNING] Python 3.8+ is highly recommended!")
        else:
            print("[PASS] Python version check passed.")

        print("-" * 50)

        # 2. Check FFmpeg / FFprobe
        ffmpeg_ok = check_command("ffmpeg")
        ffprobe_ok = check_command("ffprobe")

        if ffmpeg_ok:
            print("[PASS] FFmpeg is installed and available in PATH.")
        else:
            print("[FAIL] FFmpeg is NOT found in your PATH.")

        if ffprobe_ok:
            print("[PASS] FFprobe is installed and available in PATH.")
        else:
            print("[FAIL] FFprobe is NOT found in your PATH.")

        if not (ffmpeg_ok and ffprobe_ok):
            import platform
            print("\n" + "!" * 50)
            print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
            _sys = platform.system()
            if _sys == "Windows":
                print("Install on Windows:  winget install Gyan.FFmpeg")
            elif _sys == "Darwin":
                print("Install on macOS:    brew install ffmpeg")
            else:
                print("Install on Linux:    sudo apt-get install -y ffmpeg   (or your
distro's package manager)")
            print("Or download a build from https://ffmpeg.org/download.html and put
ffmpeg/ffprobe on PATH.")
            print("!" * 50 + "\n")
        else:
```

```
111             print("[SUCCESS] All essential media command line tools are ready.")
112
113         print("-" * 50)
114
115         # 3. Check ML / GPU (Optional but good)
116         try:
117             import torch
118             cuda_ok = torch.cuda.is_available()
119             if cuda_ok:
120                 print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")
121             else:
122                 print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
Falling back to CPU.")
123         except ImportError:
124             print("[INFO] PyTorch is not installed. Relevance checks will run using
lightweight visual heuristics.")
125
126         print("=" * 50)
127         return ffmpeg_ok and ffprobe_ok
128
129     if __name__ == "__main__":
130         init_default_config()
131         media_ok = run_diagnostics()
132         run_pip_install()
133         print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
134
```

14. assistant (2026-06-21T13:52:31.359Z)

Now I see ffmpeg/ffprobe are called as bare `"ffmpeg"`/`"ffprobe"` strings in `subprocess` (relying on PATH), and `doctor.py` already checks PATH + suggests `winget install Gyan.FFmpeg`. Let me look at how weights are downloaded from GCS, and check for any existing weight-download / model-fetch code.

15. user (2026-06-21T13:52:32.313Z)

```
1       """Google Cloud Storage backend ? first-class object buckets.
2
3       Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4       install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5       (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6       never from config. A ``client`` may be injected for tests.
7       """
8       from __future__ import annotations
9
10      from datetime import timedelta
11      from typing import Any, Iterator, Optional
12
13      from backend.storage.base import StorageBackend
14      from backend.storage.ref import StorageRef, StorageStat
15
16
17      class GCSStorage(StorageBackend):
18          def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                       signed_url_ttl: int = 3600, **_ignored) -> None:
20              self._ttl = signed_url_ttl
21              if client is not None:
22                  self._client = client
23                  return
24              from google.cloud import storage  # type: ignore  # lazy: only when a gcs:// URI
is used
```

```
            self._client = storage.Client(project=project)

        def _blob(self, ref: StorageRef):
            return self._client.bucket(ref.bucket).blob(ref.key)

        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
            import os
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            self._blob(ref).download_to_filename(dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
            return ref

        def exists(self, ref) -> bool:
            return bool(self._blob(ref).exists())

        def stat(self, ref) -> StorageStat:
            b = self._blob(ref)
            b.reload()
            return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
                               content_type=b.content_type)

        def list(self, prefix) -> Iterator[StorageRef]:
            for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
                yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")

        def delete(self, ref) -> None:
            self._blob(ref).delete()

        def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
            return self._blob(ref).generate_signed_url(
                version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)

```

16. user (2026-06-21T13:52:33.321Z)

```
Found 56 files
backend\main.py
backend\config\models.py
tests\test_api_endpoints.py
backend\worker\__main__.py
tests\test_native_wasb_runner.py
backend\pipeline\detectors\native_wasb_runner.py
tests\test_output_base.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\storage\__init__.py
training\gen0\harness.py
tests\test_worker.py
tests\test_training_gen0.py
backend\utils\workspace.py
tests\test_config_wasb.py
tests\test_storage.py
backend\storage\gdrive.py
tests\test_wasb_cache.py
artifacts\so.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
```

backend\pipeline\detectors\wasb_infer.py
scripts\doctor.py
backend\storage\s3.py
backend\storage\gcs.py
backend\pipeline\detectors\stub_runner.py
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
backend\eval\ablation.py
tests\test_upload_download.py
backend\pipeline\detectors\tracknet_runner.py
scripts\run_phase3_eval.py
tests\test_tolerance_metrics.py
tests\test_tracknet_runner.py
tests\test_golden_real_fixtures.py
tests\test_fusion_hybrid.py
backend\eval\tolerance_metrics.py
backend\pipeline\detectors\wasb_runner.py
tests\test_wsl_tilde_quoting.py
backend\pipeline\detectors\base.py
backend\eval\distill_local.py
tests\test_personalization_serving.py
tests\test_per_user_eval.py
backend\eval\fusion_golden.py
backend\utils\hashing.py
backend\eval\score_calibration.py
backend\pipeline\detectors\person_detector.py
tests\test_person_detector.py
tests\test_trajectory_hybrid_equivalence.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\reporting\harvest.py
tests\test_reporting_harvest.py
run_regression.py
tests\test_detector_keying.py
tests\test_hashing.py
run_eval.py
tests\test_eval_harness.py

17. assistant (2026-06-21T13:52:36.574Z)

Let me look at how WASB weights are referenced/downloaded (the native runner and wasb_runner)
and check the appliance architecture doc in khelsutra plus the superseded desktop plan.

18. user (2026-06-21T13:52:38.141Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8       see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14          (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15          strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
```

```
16        * weights / repo / python / scratch come from the **environment** first
17          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26      **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27      (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28      candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29      ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30      run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31      vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32      artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34      not the numbers.
35      """
36      import os
37      import shutil
38      import logging
39      import subprocess
40      from dataclasses import dataclass
41      from typing import Any, Dict, List, Optional, Tuple
42
43      from backend.pipeline.detectors.base import DetectorRunner
44      from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
45      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
46      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
47
48      logger = logging.getLogger(__name__)
49
50      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
51      # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
52      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
53
54      # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
55      # so the "cuda True/False" string they parse can never drift apart.
56      _CUDA_PROBE_CODE = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
57
58
59      @dataclass
60      class NativeWasbConfig:
61          """Resolved settings for a Linux-native WASB run.
62
63          Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
64          over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
65          defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
66          """
67          repo_dir: str = "~/models/WASB-SBDT"   # native clone of nttcom/WASB-SBDT
```

```
68          python_bin: str = "python"            # the `wasb` conda env python (invoked by
path, no activate)
69          weights_path: str = ""                # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
70          sport: str = "badminton"
71          device: str = "cuda"                  # auto | cuda | mps | cpu (auto = cuda-if-
available-else-cpu)
72          scratch_dir: str = ""                 # frame-extraction scratch (env); "" = next
to the output dir
73          timeout_sec: int = 1800              # 30 min; a hung GPU call is killed (0 = no
timeout)
74          keep_frames: bool = False             # retain the decoded frame cache after
success (debug only)
75          # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
76          # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
77          # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
78          stream_video: bool = True
79
80          @classmethod
81          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
82              w = (idx_cfg or {}).get("wasb", {}) or {}
83              env = os.environ.get
84              # stream_video is tri-state in config: None/absent => native default (stream);
an
85              # explicit True/False still wins (e.g. False for a container cv2 can't seek).
86              sv = w.get("stream_video")
87              return cls(
88                  repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
89                  python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
90                  weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
91                  sport=w.get("sport", "badminton"),
92                  device=env("WASB_DEVICE") or w.get("device", "cuda"),
93                  scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
"",
94                  timeout_sec=int(w.get("timeout_sec", 1800)),
95                  keep_frames=bool(w.get("keep_frames", False)),
96                  stream_video=(True if sv is None else bool(sv)),
97              )
98
99
100     class NativeWasbRunner(DetectorRunner):
101         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
102
103         def __init__(self, cfg: NativeWasbConfig):
104             self.cfg = cfg
105             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
106             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
107             self._cuda_available: Optional[bool] = None
108
109         # --- low-level native exec (the single place tests patch)
-------------------------
110         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
111             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
112             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
113                                   timeout=(self.cfg.timeout_sec or None))
114
115         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) ------
```

```
116         def _probe_cuda(self) -> bool:
117             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
118             (missing python / timeout / import error) is treated as 'no CUDA'."""
119             try:
120                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
121             except (FileNotFoundError, subprocess.TimeoutExpired):
122                 return False
123             return res.returncode == 0 and "cuda True" in (res.stdout or "")
124 
125         def _cuda_is_available(self) -> bool:
126             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
127             if self._cuda_available is None:
128                 self._cuda_available = self._probe_cuda()
129             return self._cuda_available
130 
131         def _effective_device(self) -> str:
132             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
133             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
134             if self.cfg.device == AUTO:
135                 return DeviceContext(AUTO, self._cuda_is_available()).effective
136             return self.cfg.device
137 
138         def _repo_src(self) -> str:
139             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
140 
141         def _copy_infer_script(self) -> bool:
142             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
143             repo_src = self._repo_src()
144             try:
145                 os.makedirs(repo_src, exist_ok=True)
146                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
147                 return True
148             except OSError as e:
149                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
150                 return False
151 
152         def _rm_rf(self, path: Optional[str]) -> None:
153             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
154             if path:
155                 shutil.rmtree(path, ignore_errors=True)
156 
157         def healthcheck(self) -> Tuple[bool, str]:
158             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
159             if not self.cfg.weights_path:
160                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
161             weights = os.path.expanduser(self.cfg.weights_path)
162             if not os.path.exists(weights):
163                 return False, f"WASB weights not found at {weights}"
164             repo_src = self._repo_src()
165             if not os.path.isdir(repo_src):
166                 return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
167                                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
168             # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
169             # argparse rc=2 from wasb_infer.py at run time.
170             if self.cfg.device not in VALID_DEVICES:
```

```
171                 return False, (f"unknown device {self.cfg.device!r} ? valid: {',
'.join(VALID_DEVICES)}.")
172             try:
173                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
174             except FileNotFoundError:
175                 return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
176             except subprocess.TimeoutExpired:
177                 return False, "native torch healthcheck timed out."
178             out = (res.stdout or "").strip()
179             if res.returncode != 0:
180                 return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
181             # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
182             self._cuda_available = "cuda True" in out
183             ctx = DeviceContext(self.cfg.device, self._cuda_available)
184             if ctx.cuda_required_but_missing:
185                 # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
186                 # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
187                 return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
188             if ctx.fell_back:
189                 logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO
CPU; WASB "
190                                "inference will be slow. Set indexing.wasb.device=cuda to
require a GPU.", out)
191             return True, out
192
193         def run_predict(self, video_win_path: str, output_win_dir: str,
194                         video_id: Optional[str] = None) -> Optional[str]:
195             """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
196
197             Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
198             runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
199             same downstream contract, so the parity comparison is apples-to-apples.
200             """
201             output_dir = os.path.abspath(output_win_dir)
202             os.makedirs(output_dir, exist_ok=True)
203             # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
204             norm = str(video_win_path).replace("\\", "/")
205             stem, _ext = os.path.splitext(os.path.basename(norm))
206             # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
207             key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
208             out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
209
210             if not self._copy_infer_script():
211                 logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
212                 return None
213
214             weights = os.path.expanduser(self.cfg.weights_path)
215             argv = [self.cfg.python_bin, "wasb_infer.py",
216                     "--video", os.path.abspath(str(video_win_path))]
217             frames_dir: Optional[str] = None
218             if self.cfg.stream_video:
219                 # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
220                 argv.append("--stream-video")
221             else:
222                 scratch = self.cfg.scratch_dir or output_dir
223                 frames_dir = os.path.join(scratch, f"{key}_frames")
```

```
224                 argv += ["--frames_out_dir", frames_dir]
225             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
226             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
227             device = self._effective_device()
228             argv += ["--weights", weights,
229                      "--sport", self.cfg.sport,
230                      "--device", device,
231                      "--out", out_csv]
232
233             logger.info("Running native WASB inference (device=%s%s) on %s ...",
234                         device, " [auto?fallback]" if self.cfg.device == AUTO and device !=
"cuda" else "",
235                         os.path.basename(norm))
236             try:
237                 res = self._run(argv, cwd=self._repo_src())
238             except FileNotFoundError:
239                 logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
240                 return None
241             except subprocess.TimeoutExpired:
242                 logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
243                 return None
244             if res.returncode != 0:
245                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
246                 return None
247             if not os.path.exists(out_csv):
248                 logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
249                 return None
250             logger.info("WASB trajectory CSV: %s", out_csv)
251
252             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
253             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
254             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
255             if not self.cfg.keep_frames and frames_dir:
256                 logger.info("Cache hygiene: removing frame cache for %s "
257                             "(set indexing.wasb.keep_frames=true to retain).", key)
258                 self._rm_rf(frames_dir)
259             return out_csv
260
261         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
262         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
263         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
264
```

19. user (2026-06-21T13:52:39.166Z)

C:\Users\avidu\Projects\khelsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md

20. user (2026-06-21T13:52:39.205Z)

docs\DESKTOP_APP_PLAN.md

21. assistant (2026-06-21T13:52:42.290Z)

Now let me read the live appliance architecture doc and the superseded desktop plan to understand what's already decided about packaging, weights, and ffmpeg.

22. user (2026-06-21T13:52:43.446Z)

```
1       # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3       > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5       > **Tracking anchors:** §9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6       > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7       > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9       ---
10
11      ## 0. TL;DR
12      **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14      The **single most important caveat:** the engine has **zero app-layer auth and CORS
`*`** `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
tenancy boundary.**
15
16      Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection
there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU
host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel**
with the UI MVP, which ships on Gemini today.
17
18      ## 1. Problem & goal
19      The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole
loop on a single box, **honestly, behind a gate**.
20
21      **Headline CUJ (owner's words):** a **hosted website** where I pick an input ? **a
Google Drive file, a bucket file (`s3://`/`gs://`), or a local upload** ? it lands in the
**bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host**
running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI.**
22
23      **Outcome ("good"):** an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host
later** (native WASB) with **no UI fork**.
24
25      **Read to get current:** `PER_RALLY_SELECTION.md` (§3/§4/§10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27      ## 2. Decisions locked
```

#	Decision	Source / date	Implication
D1	Reverse-proxied **single origin**; engine stays `127.0.0.1:8000`	`[verified main.py:638]`; 2026-06-20	CORS `*` is moot from the browser ? the proxy is the boundary
D2	**Edge auth**, zero engine auth code	`HOSTING_PLAN.md`; 2026-06-20	Caddy basicauth OR cloudflared + Cloudflare Access
D3	MVP drives the **in-process FastAPI worker** loop, `storage=local`	`[verified job_worker.py:37, local.py:33]`; 2026-06-20	bucket worker = reserved scale path, out of MVP
D4	Embed **RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim**	PR [#11]; 2026-06-20	the console is a superset; do not redesign the picker
D5	Console lives in the khelsutra **`web/`** React scaffold	`web/README.md`; 2026-06-20	Cloudflare-Pages-style root, not engine static
D6	Compute placement is a **setting**, rendered from `GET /api/capabilities`	`[design]`; 2026-06-20	UI offers ONLY what the box can honestly do
D7	**Ingest is a source menu** (local upload · bucket URI · Drive) normalizing to one video object	`[verified backend/storage/ref.py parse_uri]`; 2026-06-20	Drive is a **stub** today ? scoped honestly (§4.2)
D8	GPU host is **provider-agnostic** (DO droplet / GCE VM / Cloud Run+GPU); storage stays the bucket	Owner 2026-06-20	compute and storage are separate planes; no provider lock-in
D9	**Observability is first-class** ? structured logs on every new path + a DoD item	Owner 2026-06-20	see §5; verified during CUJ replay
D10	Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop app); the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the tool	Owner 2026-06-20	**supersedes `DESKTOP_APP_PLAN.md`**; one artifact (this appliance) runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings

3. Foundation ? what the engine gives us today `[verified this session unless noted]`

- **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ? `POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}` (FileResponse) `[main.py]`. **Chunked upload exists** ? `POST /api/upload/init` · `PUT /api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}` `[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is **client-wiring only**, no engine change.
- **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)` `[verified job_worker.py:37]` ? strictly **one job at a time**; the DB `jobs` table is the clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a **separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1 to serialize) ? it is **not** already 1.
- **Storage:** `storage.backend` default = `"local"` `[verified models.py:298]`; `LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on `src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-local**. URI scheme `parse_uri` `[verified ref.py]`: bare/`local://` ? local; `s3://` (AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub (emulated only)**. Credentials never in config ? boto3 default chain / `~/.aws` `[verified s3.py]`.
- **Compute seam:** `indexing.detector_impl` ? {`auto`=WSL, `native`=Linux-native, `stub`=CPU mock}; default `"auto"` `[verified models.py:129]`, resolved in `trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch `feat/native-wasb-runner`:** `native_wasb_runner.py` **exists** `[verified]`; **`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified :57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ? not exercisable on the droplet or in the offline suite. Treat native as **not production-verified**.
- **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ? pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` + `--set indexing.skip_ai_handoff=true` + `storage.s3`.
- **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET /api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}` `[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.

- **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md, DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run there.
- **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row `get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any **Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.

4. Design / architecture

4.1 Topology & request path

ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy / static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI `127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing site stays a **separate** systemd unit on a **separate** subdomain.

```
PUBLIC INTERNET
   ?  (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
   ?      cloudflared Tunnel + Cloudflare Access (no inbound ports)  OR  Caddy :443 + basicauth
   ?
????????????????????????????????????????????????????????????????????????????????????
? ONE BOX   firewall BLOCKS inbound 8000 + 9000/9001/9023                          ?
? (2) reverse proxy / static host (Caddy/cloudflared)                              ?
?      /, /assets/*  ? web/ React SPA      /api/*, /api/highlights/* ? :8000 ?
?            ?  ONE ORIGIN to the browser ? engine CORS '*' is irrelevant    ?
?            ?                                                                 ?
? (3) ENGINE FastAPI  (host=127.0.0.1:8000, HARD-bound [main.py:638])       ?
?      control: jobs table + ThreadPoolExecutor(max_workers=1) drain         ?
?      data: uploads.py chunked upload · /api/compile · /api/highlights      ?
?            ?                                                               ?
? (4) STORAGE = LocalStorage (default)  fetch=identity · put=no-op(src==dst) ?
?            ?                                                               ?
? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet]          ?
?                   native WASB (GPU host) · auto (owner Win+WSL)            ?
????????????????????????????????????????????????????????????????????????????????????
RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO Spaces)
   ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3://?` ? outputs/<vid>/ back
   (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll contract)
```

4.2 Ingest sources (the headline) ? one menu, one normalized video object

All sources normalize to **"a video the pipeline can read"** ? a local path (MVP) or a `StorageRef` (scale):

Source	Path today	Status
Local upload	SPA chunks the file ? `max_part_mb` via the existing `/api/upload/init|part|complete` `[verified uploads.py]` ? server path ? `POST /api/process`. (Scale: also `StorageBackend.put` to `s3://`/`gcs://`.)	**wire client only** (endpoints exist)
Bucket URI (`s3://`, `gs://`)	**`POST /api/process` accepts the bucket URI directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283 ? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the **engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs` is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket URI 400.)*	**engine: DONE (verified); SPA ingest call: design (= B-P2)**
Google Drive (`gdrive://`)	`parse_uri` recognizes it, but the gdrive backend is an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]`	**stub ? roadmap;** MVP

requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90   ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91 | Placement | Runs | Needs | Real/mock | Honest caveat |
92 |---|---|---|---|---|
93 | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker | `GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google ? **must be in consent copy**; **no per-video cost ceiling** today |
94 | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` | nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)"; demo/regression only |
95 | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96 | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ? `NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU**; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97 | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU | REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98 | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as a setting" is **design** ? selector shown **disabled** in MVP |
99
100   **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the **containerization feasibility check** above.
101
102   ### 4.4 Auth / exposure (non-negotiable invariant)
103   No engine auth, by design; the gate is external. **Invariant (all three must hold):** engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist `[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**; set `security.allowed_video_root`; rate-limit; per-user quota.
104
105   ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106   - **S1 Ingest & Process** ? the source menu (§4.2) + a segmenter picker from `/api/capabilities` ? `POST /api/process`.
107   - **S2 Progress** ? 3s poll of the free-text stage (`hashing/validating/segmenting/finalizing`), a **"one job at a time"** notice, and the restart-failure terminal state.
108   - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`): honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter. **Per-rally is ONE screen of the console.**
109   - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for **reload-survival**.
110   - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.
111
112   ### 4.6 Reuse map
113   **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing `NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the `HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units; `GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker bridge.
114
115   ## 5. Observability ? logging & monitoring `[required, first-class]`
116   Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously, with a full log-analysis pass across every new code path.** The engine already uses a centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new path.
117
118   - **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`)

plus an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ? GPU host. Log it on every line.
119   - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry / result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with the same correlation id.
120   - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must **never** look complete; every dropped clip/chunk is logged + surfaced in the UI.
121   - **Health + metrics.** Ship the missing `GET /api/health` `[HOSTING_PLAN.md:42]` and a thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a ledger exists) for the engine-mode banner + alerting.
122   - **Aggregation for transient hosts.** A scale-to-zero Cloud Run / GPU host is **ephemeral** ? logs must ship to a durable sink (the bucket or a log service), not die with the instance.
123   - **Log-analysis pass before merge (DoD item).** Every PR that adds a code path includes a check that the path logs meaningfully, and the **logs are inspected during CUJ live-replay** (§8 Layers 1?3). Ties [[feedback-launch-quality-bar]].
124
125   ## 6. Risk table
126 | Risk | Today | Mitigation |
127 |---|---|---|
128 | No app auth + CORS `*` `[main.py:42]` | open if any of {localhost-bind, firewall, gate} slips | enforce all three; **LIVE posture test** that raw `:8000` is refused from outside |
129 | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes them |
130 | Retention | `output/`+`uploads/` originals **never deleted** | sweep / keep-proxy-only before any non-owner exposure |
131 | Surprise Gemini/GPU bill | **no cost ceiling/ledger** | per-video cap; `ai_max_workers` (default 5) tunable; `max_workers=1` job serialization |
132 | Native WASB unverified | partially wired; owner-parity-gated | treat as not-production-verified until owner byte-parity on a real GPU box |
133 | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist | coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD, explicit `git add`** |
134 | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require Drive?bucket pre-step until real |
135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before promising Cloud Run/GCE |
136 | Shared-checkout collisions | active worker sessions share the engine tree | the branch-safety protocol on every engine-side PR |
137 | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted in chat or committed (cf. the **already-rotated Spaces key** exposed in chat, `DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive / `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token, revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action every time** |
138
139   ## 7. Honest limits ? what this does NOT do
140   Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not** containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the **console plumbing**, not engine rally-detection quality (a separate track).
141
142   ## 8. CUJs & live-test plan `[required]`
143   **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core); **CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters + honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145   **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146   - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing

the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure; `query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ? CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`). The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug (CUJ-3), capability honesty (CUJ-4).
147   - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main --server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema, compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148   - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1 with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption; reuse the `parity.yml` dual-host pattern.
149   - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job + headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no stacking**; coverage at/above the floor; **logs inspected during replay** (§5).
150   - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg` behavior; a multi-GB browser upload. Reported back into §9, stated plainly as **not verified** in the offline suite.
151
152   ## 9. Deliverables & progress tracker ? **source of truth**
153   Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR ships its tests + meaningful logs (§5).
154
155 | ID | Deliverable | Depends on | Gated? | Status | PR |
156 |----|-------------|------------|--------|--------|----|
157 | T0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | owner | ? | (this PR) |
158 | T1 | Mock engine + capability fixtures + browser-replay CUJ scripts (L1) | T0 | no | ? | ? |
159 | T2 | Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key (P1) | ? | owner | ? | ? |
160 | T3 | L3 live posture test (raw `:8000`/`:900x` refused from outside) | T2 | no | ? | ? |
161 | T4 | S1 ingest (local upload wired) + segmenter picker | T1 | no | ? | ? |
162 | T5 | S2 progress (poll + one-job + restart-failure) | T4 | no | ? | ? |
163 | T6 | S3 embed **RallyPicker** + compile + download | T4 | no | ? | ? |
164 | T7 | *(engine)* `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ? | ? |
165 | T8 | *(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival) | ? | coord | ? | ? |
166 | T9 | S5 Settings (config+capabilities; disabled compute/storage target) | T7,T8 | no | ? | ? |
167 | T10 | Observability pass: correlation id end-to-end + structured logs + metrics (§5) | T4 | no | ? | ? |
168 | T11 | Retention sweep for `output/`+`uploads/` | ? | owner | ? | ? |
169 | T12 | Ingest from **bucket URI** (`s3://`/`gs://`) wiring ? **engine half DONE** (`/api/process` accepts URIs, M2/[#280], `[verified]`); remaining = the SPA ingest call (= **B-P2**) | T4 | coord | ? | engine done; SPA pending |
170 | T13 | **GPU host stand-up** (provider-agnostic) + wire `detector_impl=native`; owner byte-parity (P4) | branch | ? owner | ? | ? |
171 | T14 | *(reserved)* Bucket-worker bridge ? remote-GPU "compute as a setting" (P5) | T8,T12 | ? owner | ? | ? |
172
173   **Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the start** ? owner provisions the host + token; the engine session lands native WASB; the UI MVP does **not** block on it (ships on Gemini).
174
175   ## 10. Open questions ? owner / external
176   **Blocking gates:** (1) **edge-auth** ? Caddy basicauth (self-contained) vs

cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) **retention defaults** ? auto-delete originals after compile? keep proxies+intervals only? window? (privacy/consent). (3) **per-video Gemini/GPU cost ceiling** ? what cap, hard-fail vs warn? (4) **GPU host choice** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker worth building for Cloud Run)? (5) **P5 bridge ownership** ? who builds the orchestration endpoint + how it coordinates with the active worker sessions.
177   **Nice-to-knows:** console home (web/ vs engine static); a review-playback streaming endpoint; per-user scoping timing; containerization vs runbook.
178
179   ## 11. Definition of done
180   - [ ] CUJ-1 works end to end on the droplet (Gemini **and** stub) behind the gate, via the ingest menu.
181   - [ ] Raw `:8000`/`:900x` unreachable from outside (**tested**, not assumed).
182   - [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped honestly (not faked).
183   - [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
184   - [ ] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via `/api/videos` + `/api/reels`.
185   - [ ] **Observability:** correlation id threaded end-to-end; structured logs at every new path; loud failures; logs inspected during CUJ replay.
186   - [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
187   - [ ] CI green (L2 contract + L4 regression); coverage ? floor; §9 tracker all ?/?-resolved.
188
189   ## 12. References
190   **Engine code `[verified this session]`:** `backend/main.py:12-17,42,52,307-328,331-340,342-396,638`; `backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`; `backend/api/static/app.js:88,344,373`; `backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`; `backend/config/models.py:129,183,298`; `backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86, gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py, stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter: gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`; `DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`; `DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`; `PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`, `PORTABILITY.md`, `web/README.md`.
191
192   ---
193   ### Changelog
194   - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs (FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor re-verified against source before writing (catching the native-WASB partial-wiring + `ai_max_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local), multi-provider GPU, and first-class Observability directives.
195

23. assistant (2026-06-21T13:52:46.338Z)

Now let me read the superseded desktop plan to see what packaging approach it considered, then do web research. Let me also quickly check the WASB runner config for how weights are sourced today.

24. user (2026-06-21T13:52:46.963Z)

1   # Cross-Platform Desktop App ? "Plug in your card, get your highlights"
2
3   > **Status:** `DRAFT` ? `owner-gated` (scoping doc only; **nothing is built**) · **Owner:** avidullu · **Created:** 2026-06-18 · **Last updated:** 2026-06-18
4   > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
5   > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in

`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once the owner greenlights P0).
> **Relation to existing docs:** **peer-of** [`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) (this is the **`LocalDesktop` ComputeBackend** made into a product) · **realizes the L0 tier** of [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) · **consumes** [`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (the smaller/export-clean model is one answer to the compute crux) · bound by [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) licensing guardrails.
> **Honesty note:** every claim is tagged `[verified]` (checked against code/measured), `[researched]` (needs sign-off), or `[design]` (proposed). **The entire build is `[design]`** ? this doc scopes; it does not ship. Not legal advice.

---

0. TL;DR

Package the mature local pipeline (WASB shuttle detector + improved windowing + ffmpeg stitch, already runnable fully-offline via [`tools/generate_highlights.py --skip-ai-handoff`](../tools/generate_highlights.py)) as a **cross-platform desktop app** for **non-technical recreational players**: plug in the camera's USB/SD card, click once, get back **only the highlight reels** ? fully on-device, **no upload**, **no copy of the GBs of 4K originals**. This is the product surface that fits the "local, cheap, efficient" thesis and the L0 ("nothing leaves the machine") tier of `VIDEO_LOCALITY_MODEL.md`.

The make-or-break unknown is compute on a typical enthusiast laptop ? which often has **no discrete GPU**. WASB runs at **~3.4× real-time on a laptop RTX 2070 Super** `[verified, measured]`; on CPU it is far slower, and Apple Silicon has **no CUDA** at all. **P0 is a feasibility spike that measures CPU and Apple-MPS throughput for a ~10-min clip before any app is built.** If CPU-only is unacceptable on commodity hardware, the path needs a smaller/faster owned model (ties `OWNED_MODEL_IMPLEMENTATION_PLAN`) or a tiered "fast-if-GPU, slow-but-works-if-CPU" design. **Everything past P0 is gated on that result.**

1. Problem & goal

Why now / the user. Target users are **recreational sports enthusiasts, not tech-savvy**. Their match videos are **huge** (GBs of 4K). The cloud-upload product surface fails them on every axis: slow on a typical Indian uplink (`VIDEO_LOCALITY_MODEL.md` §0: 10 GB ? ~1 h at 20 Mbps, ~4.5 h at 5 Mbps), bandwidth-costly, privacy-fraught, and it bloats both our storage and their drives. The thing the customer actually wants back is **kilobytes of timestamps ? a highlight reel** (`VIDEO_LOCALITY_MODEL.md` §1: *"timestamps are the product; the video is dead weight"*).

The concrete outcome ("good" looks like this):
1. User plugs the GoPro/phone SD card or camera USB into their laptop.
2. The app detects the card, lists the match videos on it.
3. User clicks **Generate Highlights**.
4. The app makes an on-device proxy, runs WASB detection + windowing + stitches the reel ? **100% locally**.
5. The reel(s) land in a user-chosen folder. **The originals are never copied off the card and never bloat any drive.**

No account, no upload, no cloud dependency in the default path. Gemini stays an **optional** "better brain if you want it" toggle ? never required, never bundled as a hard dependency, never a training source (`COMMERCIALIZATION` C4).

Read to get current: `VIDEO_LOCALITY_MODEL.md` (the L0/locality framing + OQ5 packaging seed), `PLATFORM_ARCHITECTURE.md` §3.2 (the ComputeBackend seam), [`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) (the detector ABC that already anticipates a Linux-native runner), `tools/generate_highlights.py` (the local entrypoint this app wraps).

2. Decisions locked

#	Decision	Source / date	Implication

D1	**Default path is 100% on-device, no upload, no Gemini.** The reel is produced from the local pipeline (`--skip-ai-handoff`).	This doc · 2026-06-18; matches `VIDEO_LOCALITY_MODEL` L0	The app must never *require* a network call. Gemini is an opt-in toggle only.
D2	**Never copy the originals.** The app reads source video in place (off the card / a chosen folder) and writes **only** reels + a tiny report to a user folder.	This doc; mirrors `generate_highlights.py` non-destructive `_atomic_publish` discipline	No "import library" that duplicates 10 GB files. Detection runs on a proxy; stitch reads the original in place.
D3	**Drop the WSL2 dependency; run the detector natively per-OS.**	This doc; enabled by the `DetectorRunner` ABC `[verified]`	Windows-native (no WSL), Linux+NVIDIA (CUDA directly), macOS (MPS or CPU). The WSL adapter stays for the current dev box but is **not** the shipped path.
D4	**Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD only** (code, weights). ffmpeg ships as an **LGPL build** (no GPL-only encoders like x264).	`COMMERCIALIZATION` (ratified 2026-06-09); `VIDEO_LOCALITY_MODEL` OQ5 (*"ffmpeg LGPL build = licensing-clean"*)	WASB-SBDT weights are MIT `[verified]` (see `tracknet-wasb-wsl2-decision`). The stitch must use openh264 (BSD) or OS hardware encoders, **not** statically-linked x264. See §3.4.
D5	**Reuse the existing pipeline verbatim; add only the shell + a native runner + device-IO.**	`backend/pipeline/detectors/base.py` reuse contract `[verified]`	No re-architecture of windowing/stitch. New code = native runner, USB/SD detection, app shell, packaging.
D6	**Feasibility is gated on P0.** No app shell, no packaging work starts until the compute spike returns a verdict.	This doc; `decision-rigor-norm` (NUMBERS INFORM, FOUNDATIONS DECIDE)	P0 is the only un-gated deliverable; P1?P3 are owner-greenlit *after* P0 numbers.

3. Foundation ? research / prior art `[design]` / `[researched]`

3.1 The compute reality (the crux) ? what we know vs must measure

- **WASB on a discrete laptop GPU is acceptable** `[verified, measured]`: on the owner's **RTX 2070 Super Max-Q (8 GB)**, WASB inference is the pipeline driver at **~3.2?3.4× real-time** (?3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.7?6.0 GB; proxy (NVENC) ~0.2× RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5× clip length. Source: `current-state.md` `COST_SCALING.md` checkpoint (6 Boxhill clips, 2026-06-18).
- **The typical enthusiast laptop has NO discrete GPU.** A 10-min 4K clip at 3.4× RT is ~34 GPU-min *with* a 2070S. On integrated graphics / CPU-only the wall-clock is unknown and could be 10×+ worse ? possibly **hours per clip**, which would kill the one-click promise.
- **Apple Silicon has no CUDA.** WASB/TrackNet weights are PyTorch; PyTorch MPS (Metal) backend *may* run them, but op-coverage and speed on the WASB model are **unverified** `[design]`. CPU fallback on Apple is also unverified.
- **Owned-model tie-in.** `OWNED_MODEL_IMPLEMENTATION_PLAN.md` already states the on-device answer: *train in Python ? **export across the boundary** (ONNX / Core ML / ExecuTorch / GGUF) ? a thin device shell runs the artifact with zero Python*, with the standing constraint to **train with export-clean ops**. A smaller, exported, quantized shuttle detector is a credible answer if the CPU/MPS numbers on the current WASB weights are bad. **This makes P0 partly a referendum on whether the desktop product needs the owned-model track.**

3.2 The de-WSL native runner is already designed-for `[verified]`

The detector layer was deliberately built to make this port low-regret. [`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) documents, verbatim, an **"Optional Linux-native path (the main de-risk goal)"**: a `LinuxNativeRunner`/`ONNXRunner` that *"Skip[s] all wsl / _stage / conda activate, load[s] weights locally (torch or onnx)? write[s] the exact same CSV format so `trajectory_to_action_windows` works verbatim."* The WSL plumbing lives in a separate `WslCommandMixin`, explicitly so a native runner *"must not inherit WSL plumbing."* The actual inference math is already isolated in [`wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (torch + the WASB-SBDT modules, emitting `Frame,Visibility,X,Y`). **So the port is: lift `wasb_infer.py` out of WSL, load weights with native torch/onnxruntime per-OS, keep the identical CSV contract.** The windowing brain (`trajectory_to_action_windows`) and stitch are untouched.

3.3 Compute backends to evaluate in P0

Path	OS	Toolkit	Status
CUDA-native	Windows-native, Linux+NVIDIA	torch+CUDA (no WSL)	proven in WSL

`[verified]`; native = port |
57 | Apple MPS | macOS (Apple Silicon) | torch MPS / Core ML export | **unverified ? P0
must measure** |
58 | CPU | all (the no-GPU laptop) | torch-CPU / **onnxruntime** (often fastest on CPU) |
unverified ? P0 must measure; the gating number |
59 | Smaller/exported model | all | ONNX/Core ML/ExecuTorch, quantized | deferred to owned-
model track; P0 informs whether it's *needed* |
60
61   ### 3.4 App-shell prior art & licensing landscape `[researched]`
62   - **App shell options:** **Tauri** (Rust + system webview; ~3?10 MB binaries;
MIT/Apache) vs **Electron** (bundled Chromium; ~100?150 MB; MIT) vs **CLI + installer** (wrap
`generate_highlights.py`; smallest effort). All shell licenses are clean.
63   - **Backend packaging:** the Python/FastAPI backend bundles via **PyInstaller** or a
frozen venv as a local "sidecar." Packaging was already modernized ? **PEP 621 `pyproject.toml`
+ an `indexer` console entry point** shipped (`DEFERRED_HARDENING_PLAN` #7, DONE, PR #167)
`[verified]` ? so a CLI MVP has a real head start.
64   - **ffmpeg licensing (a genuine landmine):** ffmpeg is LGPL/GPL. The current stitch path
uses **libx264 (GPL)**. Redistributing that inside an installer would impose GPL on the bundle.
Mitigation (D4): ship an **LGPL ffmpeg build** using **openh264 (BSD)** or the OS hardware
encoder (NVENC / Apple **VideoToolbox** / Linux **VAAPI**) ? the desktop app should prefer
hardware H.264 anyway (faster, no CPU stitch tax).
65   - **Weights:** WASB-SBDT = MIT `[verified]` (`tracknet-wasb-wsl2-decision`),
redistributable.
66
67   ## 4. Design / architecture `[design]`
68
69   ### 4.1 The one-click flow (and what code each step reuses)
70   ```
71   [card inserted]
72   ? (NEW) per-OS device-insert watcher ? mount path
73   ?
74   [list videos on the card]   ? glob video exts; reuse _VIDEO_EXTS from
generate_highlights.py
75   ?
76   [make on-device proxy]   ? ffmpeg downscale (hardware encoder); reuse the proxy +
--detect-from split (#205)
77   ?
78   [detect: WASB native]   ? (NEW) NativeWasbRunner (DetectorRunner subclass) ? NO WSL;
same CSV contract
79   ?
80   [windowing ? rallies]   ? REUSE trajectory_to_action_windows + improved milestone
windowing (R1: cap=6 + R4)
81   ?
82   [stitch reel from ORIGINAL] ? REUSE VideoCompiler
(backend/pipeline/stitcher/compiler.py); read original in place
83   ?
84   [export reel + tiny report to a user folder]   ? reuse _atomic_publish non-destructive
write; NEVER copy originals
85   ```
86   The detector runs on a **proxy**; the stitch reads the **original on the card** (the
`--detect-from` split, already shipped `[verified]` in `generate_highlights.py`). Rally
timestamps transfer 1:1 because the proxy shares the timeline (the collector frame-identical
proxy contract, `VIDEO_LOCALITY_MODEL` §1).
87
88   ### 4.2 Reuse map ? what's NEW vs REUSED
89 | Concern | Status | Code |
90 |---|---|---|
91 | Trajectory?rally windowing | **REUSE** `[verified]` |
`tracknet_runner.trajectory_to_action_windows` (model-agnostic staticmethod) |
92 | Stitch / reel compile | **REUSE** `[verified]` |
`backend/pipeline/stitcher/compiler.py::VideoCompiler` (pure ffmpeg) |
93 | Local orchestration (proxy?detect?stitch, non-destructive, resumable, `--skip-ai-
handoff`, `--detect-from`) | **REUSE** `[verified]` | `tools/generate_highlights.py` |
94 | Detector runner contract | **REUSE** `[verified]` |
`backend/pipeline/detectors/base.py::DetectorRunner` ABC |

95 | WASB inference math | **REUSE (lift out of WSL)** `[verified]` |
`backend/pipeline/detectors/wasb_infer.py` |
96 | **Native (de-WSL) WASB runner, per-OS** | **NEW** `[design]` | `NativeWasbRunner` ?
sibling of `wasb_runner.py`, no `WslCommandMixin` |
97 | **USB/SD insert detection + video listing** | **NEW** `[design]` | per-OS: Win
WMI/`WM_DEVICECHANGE`, macOS DiskArbitration, Linux udev/udisks2 |
98 | **App shell / one-click UI** | **NEW** `[design]` | Tauri vs Electron vs CLI (§3.4; P2
decides) |
99 | **Packaging + code-signing per OS** | **NEW** `[design]` | PyInstaller/MSIX (Win),
.dmg+notarize (mac), AppImage/Flatpak (Linux) ? P3 |
100
101   ### 4.3 Where this sits in the platform architecture
102   This **is** the `local` / `LocalDesktop` **ComputeBackend** of
`PLATFORM_ARCHITECTURE.md` §3.2, realized as a shippable product instead of a server worker. The
control-plane/data-plane split degenerates cleanly to "all on one machine." Nothing here
forecloses the hosted SaaS path ? it's the privacy-max end of the same spectrum, and a genuine
differentiator (no competitor offers true on-device compute except SwingVision, which proves
it's commercially viable ? `PLATFORM_ARCHITECTURE` §intro).
103
104   ### 4.4 App-shell recommendation (to confirm at P2)
105   Lean: **start with a CLI + installer MVP** (wraps the existing
`indexer`/`generate_highlights.py` entry point ? fastest way to validate packaging, signing,
USB-detect, and the native runner end-to-end on all three OSes), then **graduate to a Tauri
GUI** for the non-technical user (tiny download fits the "huge-video laptops, slim app" frame;
the Python backend runs as a sidecar the Tauri front-end calls over localhost). Electron is the
fallback if webview inconsistencies bite. This mirrors the repo's "simplest thing first,
graduate on evidence" bias.
106
107   ## 5. Risk table `[design]`
108 | Risk | Severity | Mitigation / where decided |
109 |---|---|---|
110 | **CPU/MPS too slow on a no-GPU laptop** (kills the one-click promise) | **make-or-
break** | **P0 spike measures it before anything is built** (§6, §7-P0). Fallbacks: tiered
fast/slow UX, or the owned smaller model. |
111 | ffmpeg GPL contamination via libx264 in the installer | high (legal) | D4: LGPL build
+ openh264/hardware encoders only. |
112 | Code-signing friction (mac notarization mandatory; Win SmartScreen without EV cert) |
medium | P3 scopes certs/notarization per OS; budget the Apple Developer Program + a Windows
cert. |
113 | Privacy regressions (the on-device promise is the *selling point*) | medium | D1/D2:
no upload, no original-copy, no required network. Make the "nothing leaves your machine"
guarantee testable (assert no outbound calls in default mode). |
114 | Per-OS device-insert flakiness / removable-media edge cases (card pulled mid-run) |
medium | P1/P2 handle gracefully; never write to the card; fail safe. |
115 | Support burden for non-technical users (drivers, "it didn't detect my card") | medium
| favors a polished GUI + clear errors (reuse the actionable-error discipline already in the WSL
adapter). |
116
117   ## 6. Honest limits ? what this does NOT do
118   - **A DRAFT/DONE status here proves nothing about runtime feasibility.** The doc is a
scope; only the P0 numbers decide whether the product is buildable on commodity hardware.
119   - **If CPU-only is too slow on a typical laptop, the simple "wrap the pipeline" path
does not exist** ? it becomes "ship a smaller owned model" (a much larger effort, ties
`OWNED_MODEL_IMPLEMENTATION_PLAN`) or "GPU-tier only" (shrinks the addressable market). The doc
names this honestly rather than assuming the wrap is free.
120   - **No accuracy claim changes.** The desktop app inherits exactly the current rally-
detection quality (R1 milestone: precision is still the open gap per
`RALLY_DETECTION_GAP_CLOSURE.md`); packaging does not improve detection.
121   - **Gemini-quality reels need the optional online toggle.** The fully-local default uses
`--skip-ai-handoff`; users wanting the Gemini brain accept that ?1280px chunks leave the machine
(consent language, `VIDEO_LOCALITY_MODEL` OQ3).
122   - **macOS support is contingent on P0's MPS/CPU verdict** ? it is not assumed.
123
124   ## 7. Deliverables & progress tracker   ? **source of truth**
125   Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,

branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands. **Everything below is owner-gated; only this doc (the scoping PR) is in flight today.**

ID	Deliverable	Depends on	Gated?	Status	PR
D0	**This scoping doc** (`DESKTOP_APP_PLAN.md` + README index)	?	No	?	*(this PR)*
P0	**Compute-feasibility spike (make-or-break).** Measure WASB throughput for a ~10-min clip on: (a) CPU-only (torch-CPU & onnxruntime), (b) Apple MPS, (c) confirm CUDA-native (no WSL). Output: a wall-clock table + a go/no-go verdict + "do we need a smaller model?" recommendation. **No app code.**	D0	**Owner greenlight**	?	?
P1	**De-WSL native detector port** (`NativeWasbRunner`, per-OS): CUDA-native (Win/Linux), MPS/CPU fallback, identical `Frame,Visibility,X,Y` CSV; reuse windowing verbatim; healthcheck per backend.	P0 verdict	Yes	?	?
P2	**Desktop app shell + one-click flow**: USB/SD detect ? list ? proxy ? native detect ? stitch ? export; shell choice (CLI MVP ? Tauri) confirmed; "no upload / no original-copy" enforced & testable.	P1	Yes	?	?
P3	**Packaging + code-signing installers** per OS (Win MSIX/Authenticode, mac .dmg + Developer-ID notarization, Linux AppImage/Flatpak); LGPL ffmpeg bundling; MIT/Apache/BSD audit of the bundle.	P2	Yes	?	?

8. Open questions ? owner / external

Blocking gates (owner decides before the dependent phase):

1. **(gates P1?P3) Greenlight P0?** ? the spike is cheap and decides everything. Recommend: yes, run P0 next.
2. **(gates P1) If CPU is too slow, which fork?** tiered GPU-fast/CPU-slow UX · invest in a smaller exported owned model (`OWNED_MODEL_IMPLEMENTATION_PLAN`) · GPU-tier-only product (narrower market).
3. **(gates P2) App shell:** confirm CLI-MVP-then-Tauri, or go straight to a GUI / Electron.
4. **(gates P3) Distribution & signing budget:** Apple Developer Program (mandatory for notarization), a Windows code-signing cert (EV for clean SmartScreen) ? owner cost decision.

Nice-to-knows:

5. OS coverage priority for v1 (Windows-first? given the owner's box and the academy beachhead).
6. Auto-update strategy (Tauri updater / Sparkle / none for the pilot).
7. Does the desktop app reuse the optional Gemini toggle at all, or stay strictly local for v1?

9. Definition of done

This subproject is DONE (? flip Status, archive) when:

- [ ] **P0** produced a measured throughput table (CPU / MPS / CUDA-native) and an owner-accepted go/no-go verdict (incl. whether a smaller model is required). *(P0 alone is a valid stopping point if the verdict is "not feasible without the owned model" ? then this doc hands off to `OWNED_MODEL_IMPLEMENTATION_PLAN`.)*
- [ ] **P1** the native (de-WSL) detector runs on ?1 GPU OS and ?1 CPU/MPS path, producing byte-compatible trajectories (regression vs the WSL runner on a shared clip).
- [ ] **P2** a non-technical user can insert a card and get a reel with no terminal, no upload, and no copied original ? verified end-to-end on ?1 OS.
- [ ] **P3** signed/notarized installers exist for the greenlit OSes; the bundle passes an MIT/Apache/BSD (+ LGPL-ffmpeg) license audit.
- [ ] §7 rows all ? and the headline feasibility verdict is recorded for posterity.

10. References

Internal code anchors `[verified]`:

[`tools/generate_highlights.py`](../tools/generate_highlights.py) (local orchestration; `--skip-ai-handoff`, `--wasb-stream`, `--detect-from`, `_WSL_DETECTORS`, `_atomic_publish`) · [`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) (`DetectorRunner` ABC + the documented Linux-native de-risk path + `WslCommandMixin`) · [`backend/pipeline/detectors/wasb_runner.py`](../backend/pipeline/detectors/wasb_runner.py) (the WSL adapter to replace) · [`backend/pipeline/detectors/wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (the torch inference core to lift native) · [`backend/pipeline/detectors/tracknet_runner.py`](../backend/pipeline/detectors/tracknet_runner.py) (shared `trajectory_to_action_windows`) ·

[`backend/pipeline/segmenters/wasb_hybrid.py`](../backend/pipeline/segmenters/wasb_hybrid.py) · `backend/pipeline/stitcher/compiler.py::VideoCompiler` · `pyproject.toml` + `indexer` console entry (#167).
157   **Internal docs:** [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) (L0 fully-local tier; OQ5 packaging) · [`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) §3.2 (ComputeBackend seam) · [`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (export-clean on-device model) · [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (C4 paid-LLM no-train; MIT/Apache/BSD-only) · [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) (#7 packaging, DONE) · [`TRACKNET_WSL_SETUP.md`](TRACKNET_WSL_SETUP.md) (the WSL setup being superseded for shipped builds).
158   **Measured source:** `current-state.md` 2026-06-18 `COST_SCALING.md` checkpoint (WASB ~3.4× RT on RTX 2070 Super Max-Q, peak VRAM ~5.7?6.0 GB).
159   **External (to validate at P0/P2/P3):** PyTorch MPS backend · onnxruntime (CPU/CoreML/DirectML execution providers) · Tauri vs Electron sizing · ffmpeg LGPL build / openh264 · Apple Developer-ID notarization · Windows Authenticode/MSIX.
160
161   ---
162
163   ### Changelog
164   - `2026-06-18` ? Created. Scoping doc for the cross-platform desktop app + the de-WSL native-compute port; all `[design]`. §7 P0 (compute-feasibility spike) named the make-or-break gate. Owner-gated; nothing built.
165

25. assistant (2026-06-21T13:52:53.847Z)

I have strong grounding in the codebase now. The key facts:
- ffmpeg/ffprobe invoked as bare strings via subprocess (PATH-dependent) in `backend/utils/video.py` and the stitcher compiler.
- `scripts/doctor.py` already checks PATH and suggests `winget install Gyan.FFmpeg`.
- WASB weights sourced from filesystem path/env (`WASB_WEIGHTS`), with GCS only as a generic StorageBackend.
- Prior decisions: D4 (LGPL ffmpeg, openh264, no GPL x264), WASB-SBDT weights are MIT.

Now let me do the web research on the load-bearing claims: imageio-ffmpeg licensing/bundle, BtbN vs gyan.dev builds, GPL vs LGPL redistribution, Hugging Face Hub download, PyPI wheel size limits, huggingface_hub licensing.

Let me load the web tools.

26. user (2026-06-21T13:53:02.234Z)

Web search results for query: "imageio-ffmpeg bundled ffmpeg binary license LGPL or GPL build"

Links: [{"title":"FFmpeg License and Legal Considerations","url":"https://www.ffmpeg.org/legal.html"},{"title":"FFmpeg: License","url":"https://ffmpeg.org/doxygen/4.4/md_LICENSE.html"},{"title":"FFmpeg Commercial License Guide: LGPL, GPL & Patent Risks | 32blog","url":"https://32blog.com/en/ffmpeg/ffmpeg-commercial-license-guide"},{"title":"FFmpeg/LICENSE.md at master · FFmpeg/FFmpeg","url":"https://github.com/FFmpeg/FFmpeg/blob/master/LICENSE.md"},{"title":"FFmpeg: LICENSE","url":"https://www.ffmpeg.org/doxygen/2.6/md_LICENSE.html"}]

Based on the search results, here's what I found regarding FFmpeg licensing:

FFmpeg can be used commercially if you build it under the LGPL (the default license) and avoid enabling GPL or nonfree options. GPL is disabled by default?simply don't pass --enable-gpl and your build stays LGPL.

However, the licensing situation is more nuanced: Building FFmpeg with --enable-gpl makes the entire FFmpeg build GPL, and embedding that build in a closed-source commercial application is not permitted.

Additionally, FFmpeg can be combined with a number of external libraries, which sometimes affect the licensing of binaries resulting from the combination.

For the specific case of the `imageio-ffmpeg` package and how its bundled binary was built, you would need to check the package's documentation or build configuration to determine whether the FFmpeg binary was compiled with GPL options enabled or kept under LGPL.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

27. user (2026-06-21T13:53:02.861Z)

Web search results for query: "BtbN ffmpeg-builds GPL vs LGPL static Windows builds redistribution"

Links: [{"title":"GitHub - BtbN/FFmpeg-Builds · GitHub","url":"https://github.com/BtbN/FFmpeg-Builds"},{"title":"Releases · BtbN/FFmpeg-Builds","url":"https://github.com/BtbN/FFmpeg-Builds/releases"},{"title":"What's the difference between ffmpeg gpl version and lgpl version? · Issue #29 · BtbN/FFmpeg-Builds","url":"https://github.com/BtbN/FFmpeg-Builds/issues/29"},{"title":"Ragerworks","url":"https://winget.ragerworks.com/package/BtbN.FFmpeg.GPL"},{"title":"Install FFmpeg (GPL static variant, 7.1 release branch) ...","url":"https://ragerworks.com/package/BtbN.FFmpeg.GPL.7.1"},{"title":"FFmpeg License and Legal Considerations","url":"https://www.ffmpeg.org/legal.html"},{"title":"FFmpeg Commercial License Guide: LGPL, GPL & Patent Risks | 32blog","url":"https://32blog.com/en/ffmpeg/ffmpeg-commercial-license-guide"}]

Based on the search results, here's what I found about BtbN FFmpeg-Builds regarding GPL vs LGPL static Windows builds and redistribution:

Build Variants

BtbN offers a GPL static FFmpeg build that includes all dependencies, even those that require full GPL instead of just LGPL. There are also gpl-shared and lgpl-shared variants available, where the lgpl-shared option comes with the lgpl set of dependencies.

Redistribution Requirements

For understanding the redistribution implications of these different builds:

LGPL Builds: Static linking is possible under LGPL, but you must provide the object files or source code necessary for the user to re-link against a modified version of FFmpeg, though dynamic linking is simpler in practice.

GPL Builds: Building FFmpeg with --enable-gpl makes the entire FFmpeg build GPL, and embedding that build in a closed-source commercial application is not permitted.

The key difference is that GPL builds have stronger copyleft requirements that extend to your entire application, while LGPL builds allow proprietary applications but with specific redistribution obligations regarding source code access for re-linking.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

28. assistant (2026-06-21T13:53:05.903Z)

Let me get the precise imageio-ffmpeg licensing details and the huggingface_hub package license / download mechanics.

29. user (2026-06-21T13:53:12.376Z)

imageio-ffmpeg License and Build Information

Package License

The imageio-ffmpeg Python package itself is licensed under the **BSD-2-Clause license**, as indicated in the repository.

Bundled FFmpeg Binary Details

The provided GitHub page content does not include specific information about:
- Whether the bundled ffmpeg binary is GPL or LGPL licensed
- What specific codecs/encoders are included (such as libx264)
- Whether ffprobe is bundled alongside ffmpeg
- The exact build configuration details

Supported Platforms

The documentation states the wheels support: "Windows 7+, Linux kernel 2.6.32+, OSX 10.9+" and that "any version of Python 3.7+ (including Pypy)" is compatible.

Recommendation

For detailed information about the ffmpeg binary's license (GPL vs LGPL), codec inclusion, and build specifics, you would need to consult the project's documentation, releases page, or contact the maintainers directly, as these technical details are not provided in the current GitHub page content.

30. user (2026-06-21T13:53:15.299Z)

Web search results for query: "huggingface_hub python package license Apache hf_hub_download offline HF_HUB_OFFLINE cache"

Links: [{"title":"huggingface-hub · PyPI","url":"https://pypi.org/project/huggingface-hub/"},{"title":"GitHub - huggingface/huggingface_hub: The official Python client for the Hugging Face Hub. · GitHub","url":"https://github.com/huggingface/huggingface_hub"},{"title":"Releases · huggingface/huggingface_hub","url":"https://github.com/huggingface/huggingface_hub/releases"},{"title":"huggingface_hub/src/huggingface_hub/file_download.py at main · huggingface/huggingface_hub","url":"https://github.com/huggingface/huggingface_hub/blob/main/src/huggingface_hub/file_download.py"},{"title":"Ultimate guide to huggingface_hub library in Python","url":"https://deepnote.com/blog/ultimate-guide-to-huggingfacehub-library-in-python"},{"title":"Downloading files · Hugging Face","url":"https://huggingface.co/docs/huggingface_hub/en/package_reference/file_download"},{"title":"Command Line Interface (CLI) · Hugging Face","url":"https://huggingface.co/docs/huggingface_hub/main/en/guides/cli"},{"title":"huggingface_hub/src/huggingface_hub/_snapshot_download.py at main · huggingface/huggingface_hub","url":"https://github.com/huggingface/huggingface_hub/blob/main/src/huggingface_hub/_snapshot_download.py"}]

Based on the search results, here's information about the huggingface_hub Python package:

License

The project is open-source under the Apache 2.0 license.

hf_hub_download Function

The `hf_hub_download` function can be used to download files from the Hub, with example usage: `hf_hub_download(repo_id="tiiuae/falcon-7b-instruct", filename="config.json")`. Files will be downloaded in a local cache folder.

Offline Mode and HF_HUB_OFFLINE

The search results show references to offline functionality. The function raises a ValueError if `force_download=True` when offline mode is enabled, with an error message stating "Cannot find the requested files in the disk cache and outgoing traffic has been disabled. To enable hf.co look-ups and downloads online, set 'local_files_only' to False."

Additionally, the environment variable `HF_HUB_OFFLINE=0` can be used to enable offline mode, and the CLI diagnostic output shows that `HF_HUB_OFFLINE` is a configurable setting for the

package.

Caching

Internally, huggingface_hub manages caching (storing downloaded files under a cache directory with content hashing for versioning) and error handling. The caching system prevents re-downloading files, was updated in v0.8.0 to become the central cache-system shared across libraries that depend on the Hub, and is documented in the cache-system guide.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

31. assistant (2026-06-21T13:53:19.345Z)

Let me get precise details on the imageio-ffmpeg binary build (the README "About" section explains the build config), and PyPI file size limits.

32. user (2026-06-21T13:53:27.717Z)

Web search results for query: "PyPI maximum file size limit per project release default 100 MiB request increase"

Links: [{"title":"Storage Limits - PyPI Docs","url":"https://docs.pypi.org/project-management/storage-limits/"},{"title":"warehouse/docs/user/project-management/storage-limits.md at main · pypi/warehouse","url":"https://github.com/pypi/warehouse/blob/main/docs/user/project-management/storage-limits.md"},{"title":"Documentation for pypi upload limits? - Packaging - Discussions on Python.org","url":"https://discuss.python.org/t/documentation-for-pypi-upload-limits/9523"},{"title":"Distributing Large Files with PyPI Packages - Dampfkraft","url":"https://www.dampfkraft.com/code/distributing-large-files-with-pypi.html"},{"title":"Pypi package size limit · Issue #86 · pypa/packaging-problems","url":"https://github.com/pypa/packaging-problems/issues/86"},{"title":"60 MB file size limit on PyPI. Modify the build_distrib.py tool to reduce packages size. · Issue #321 · cztomczak/cefpython","url":"https://github.com/cztomczak/cefpython/issues/321"},{"title":"limits · PyPI","url":"https://pypi.org/project/limits/"},{"title":"pypi: exceeded maximum storage limit of 10 GB · Issue #5622 · valhalla/valhalla","url":"https://github.com/valhalla/valhalla/issues/5622"},{"title":"Increase our project limit size · Issue #5123 · pypi/warehouse","url":"https://github.com/pypi/warehouse/issues/5123"}]

Based on the search results, here's the information about PyPI file size limits:

Default Limits

By default, PyPI limits the size of individual files to 100.0 MB. Additionally, by default, PyPI limits the total size of all files in a project to 10.0 GB.

Requesting an Increase

If you can't upload your project's release to PyPI because you're hitting the upload file size limit, PyPI can sometimes increase your limit. Make sure you've uploaded at least one release for the project that's under the limit (a developmental release version number is fine). Then, file an issue and tell them a brief description of your project, including the reason for the additional size.

The official PyPI documentation on storage limits can be found at the [PyPI Docs - Storage Limits](https://docs.pypi.org/project-management/storage-limits/) page.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

33. user (2026-06-21T13:53:28.776Z)

FFmpeg Binary Information in imageio-ffmpeg

Based on the PyPI page, the package provides minimal details about the bundled ffmpeg binary:

What's Stated

License and Bundling: The page notes that "platform-specific wheels contain the binary executable of ffmpeg, which makes this package around 60 MiB in size."

Size Details: The actual wheel sizes vary by platform?ranging from 19.7 MB (Windows 32-bit) to 31.2 MB (Windows 64-bit).

What's Not Disclosed

The PyPI listing does **not specify**:
- Whether the ffmpeg build uses GPL or LGPL licensing
- Which codecs or encoders are included (H.264, x264, libx264, openh264, etc.)
- Whether ffprobe is bundled
- Whether the binary is pre-packaged in wheels or downloaded on first use

For these technical details about the ffmpeg build configuration, you would need to consult the project's GitHub repository (https://github.com/imageio/imageio-ffmpeg), as the PyPI page itself contains no such information.

34. assistant (2026-06-21T13:53:33.485Z)

Let me get the authoritative imageio-ffmpeg build config from the README (it documents how the binary is built and from which source) and confirm the gyan.dev build licensing.

35. user (2026-06-21T13:53:38.563Z)

Based on the provided content, I cannot find the requested information. The document does not include sections describing:

- How the bundled ffmpeg binary is built
- Its license or build configuration
- Which specific encoders it includes (H.264, x264, openh264, etc.)
- Whether ffprobe is included
- Details about obtaining the binary through wheels or downloads
- Any notes about it being a minimal build

The page only mentions that "The wheels on Pypi include the ffmpeg executable for all common platforms (Windows 7+, Linux kernel 2.6.32+, OSX 10.9+)" but provides no technical details about the binary's construction, capabilities, or licensing.

36. user (2026-06-21T13:53:46.533Z)

Web search results for query: "gyan.dev ffmpeg builds windows essentials full GPL license x264 winget Gyan.FFmpeg"

Links: [{"title":"Builds - CODEX FFMPEG @ gyan.dev","url":"https://www.gyan.dev/ffmpeg/builds/"},{"title":"FFmpeg (Essentials Build) Gyan - wingetCollections","url":"https://winget.ragerworks.com/package/Gyan.FFmpeg.Essentials"},{"title":"Install FFmpeg (Essentials Build) with WinGet | winstall","url":"https://winstall.app/apps/Gyan.FFmpeg.Essentials"},{"title":"Ragerworks","url":"https://ragerworks.com/package/Gyan.FFmpeg.Essentials"},{"title":"How to install FFmpeg on Windows: a complete guide | Transloadit","url":"https://transloadit.com/devtips/how-to-install-ffmpeg-on-windows-a-complete-guide/"},{"title":"FFmpeg Installation Guide for Windows / macOS / Linux","url":"https://ffmpeg-cookbook.com/en/articles/install-ffmpeg/"},{"title":"winget-pkgs/manifests/g/Gyan/FFmpeg/5.1.2/Gyan.FFmpeg.locale.en-US.yaml at master · microsoft/winget-pkgs","url":"https://github.com/microsoft/winget-pkgs/blob/master/manifests/g/Gyan/FFmpeg/5.1.2/Gyan.FFmpeg.locale.en-US.yaml"},{"title":"FFmpeg Windows builds at gyan.dev - VideoHelp

Forum","url":"https://forum.videohelp.com/threads/399117-FFmpeg-Windows-builds-at-gyan-dev"},{"title":"Install FFmpeg 8.1.1 with winget | Gyan | Wingetly","url":"https://www.wingetly.io/apps/gyan/ffmpeg"},{"title":"Download and install FFmpeg with winget","url":"https://winget.run/pkg/Gyan/FFmpeg"}]

Based on the search results, here's what I found about gyan.dev FFmpeg builds for Windows:

Overview

Gyan.dev FFmpeg builds are available through package managers including winget with the command: `winget install "FFmpeg (Essentials Build)"` for the essentials build or `winget install ffmpeg` for the full release.

Build Variants

The essentials build variant contains commonly used libraries, whereas the full build variant also contains most of the remainder. The essentials build includes essential libraries for video compression (e.g., libx264, libx265) and audio processing (e.g., libmp3lame, libopus).

Licensing

All builds are 64-bit, static and licensed as GPLv3. FFmpeg is distributed under GPL-3.0, and users should refer to the publisher for full license terms.

Installation Options

The release essentials build can be installed via `winget install "FFmpeg (Essentials Build)"`, the release full build via `winget install ffmpeg`, and the release full shared variant via `winget install "FFmpeg (Shared)"`.

System Requirements

The build installed via winget is the same as the one from gyan.dev (as of 2026).

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

37. assistant (2026-06-21T13:53:52.496Z)

Important finding: the gyan.dev/BtbN builds suggested by `doctor.py` (`winget install Gyan.FFmpeg`) are **GPLv3** (include x264/x265). That's a redistribution landmine if bundled, but fine if the user installs it themselves (system-install path ? the user, not us, "downloads" GPL ffmpeg; no redistribution by us). Let me confirm the imageio-ffmpeg binary build config from the source repo's resources/ or docs, and verify the BtbN LGPL build's encoder set (specifically whether it has any H.264 encoder).

38. user (2026-06-21T13:54:00.681Z)

FFmpeg Build License and Configuration Information

Based on the provided GitHub issues page content, I found **no discussion about the ffmpeg build license, whether it's GPL or LGPL, details about included encoders like libx264, or information about ffprobe inclusion**.

The issues listed include topics like video frame counting performance, FPS rounding, webcam capture, and Docker compatibility, but none address the specific build configuration or licensing questions you're asking about.

To find this information, you would need to:
- Check the project's README or documentation files
- Review the setup.py or packaging configuration

- Consult the official imageio-ffmpeg repository documentation or release notes
- Contact the maintainers directly through GitHub discussions or issues

39. user (2026-06-21T13:54:02.762Z)

Web search results for query: "imageio-ffmpeg bundled binary minimal build no ffprobe libx264 included encoder GPL"

Links: [{"title":"GitHub - imageio/imageio-ffmpeg: FFMPEG wrapper for Python · GitHub","url":"https://github.com/imageio/imageio-ffmpeg"},{"title":"imageio.plugins.ffmpeg ? imageio 2.10.0 documentation","url":"https://imageio.readthedocs.io/en/v2.10.0/reference/_backends/imageio.plugins.ffmpeg.html"},{"title":"ffprobe · Issue #23 · imageio/imageio-ffmpeg","url":"https://github.com/imageio/imageio-ffmpeg/issues/23"},{"title":"imageio.plugins.ffmpeg ? imageio 2.37.3 documentation","url":"https://imageio.readthedocs.io/en/stable/_autosummary/imageio.plugins.ffmpeg.html"},{"title":"Builds - CODEX FFMPEG @ gyan.dev","url":"https://www.gyan.dev/ffmpeg/builds/"},{"title":"This gist will show you how to build a minimalist, statically-linked ffmpeg binary under the ~/bin subdirectory on your home on Ubuntu 16.04LTS. · GitHub","url":"https://gist.github.com/hieblmedia/33772066b94b14981f118acf6f02c393"},{"title":"imageio-ffmpeg · PyPI","url":"https://pypi.org/project/imageio-ffmpeg/"}]

Based on the search results, here's what I found regarding imageio-ffmpeg and the specifications you mentioned:

The default codec used by imageio-ffmpeg is "libx264" for .mp4 files (if available from the ffmpeg executable) or "msmpeg4" for .wmv.

Regarding the bundled binary characteristics:

- **libx264 encoder**: Building with libx264 support requires ffmpeg to be configured with --enable-gpl and --enable-libx264, which indicates that GPL licensing is required when libx264 is included.

- **ffprobe**: There has been discussion about adding ffprobe support to imageio-ffmpeg for metadata-related functionality, though it appears this may not have been included in standard builds by default.

- **Minimal builds**: FFmpeg binaries can be built to include only a select set of components and libraries, with variants like the essentials build containing commonly used libraries versus full build variants containing most of the remainder.

The search results indicate that imageio-ffmpeg uses ffmpeg as a wrapper, and while libx264 is available as a default codec, the specific configuration of the bundled binary (minimal build without ffprobe) would depend on how the particular version of imageio-ffmpeg was compiled.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

40. assistant (2026-06-21T13:54:07.183Z)

Two critical findings confirmed: (1) imageio-ffmpeg defaults to libx264 for mp4 which means its bundled binary is a GPL build (libx264 requires --enable-gpl), and (2) imageio-ffmpeg does NOT bundle ffprobe (issue #23) ? and the engine's `video.py::get_video_duration` calls `ffprobe`. Both are problems for our use case. Let me confirm the ffprobe issue and check the imageio-ffmpeg `get_ffmpeg_exe()` discovery behavior, plus verify how the engine could point its bare `"ffmpeg"` calls at a discovered binary.

41. user (2026-06-21T13:54:11.648Z)

Based on the provided content, I cannot determine whether imageio-ffmpeg bundles ffprobe or only ffmpeg, nor can I find any maintainer statements about ffprobe support.

The page shows GitHub Issue #23, titled "ffprobe," where a user states: "It would be handy to

use FFPROBE for metadata." The issue references a related discussion in the MoviePy project but provides no resolution, maintainer response, or official stance on ffprobe inclusion or support.

The issue appears to remain open without closure or documented maintainer commentary in the content provided.

42. user (2026-06-21T13:54:13.652Z)

Web search results for query: "imageio_ffmpeg get_ffmpeg_exe IMAGEIO_FFMPEG_EXE environment variable override path"

Links: [{"title":"GitHub - imageio/imageio-ffmpeg: FFMPEG wrapper for Python · GitHub","url":"https://github.com/imageio/imageio-ffmpeg"},{"title":"RuntimeError: No ffmpeg exe could be found. Install ffmpeg on your system, or set the IMAGEIO_FFMPEG_EXE environment variable. · Issue #1158 · Zulko/moviepy","url":"https://github.com/Zulko/moviepy/issues/1158"},{"title":"Imageio environment variables ? imageio 2.8.0 documentation","url":"https://imageio.readthedocs.io/en/v2.8.0/envvariables.html"},{"title":"Imageio environment variables ? imageio 2.9.0 documentation","url":"https://imageio.readthedocs.io/en/v2.9.0/envvariables.html"},{"title":"imageio-ffmpeg/imageio_ffmpeg/_utils.py at main · imageio/imageio-ffmpeg","url":"https://github.com/imageio/imageio-ffmpeg/blob/main/imageio_ffmpeg/_utils.py"},{"title":"Imageio environment variables ? imageio 2.4.1 documentation","url":"https://imageio.readthedocs.io/en/v2.4.1/envvariables.html"},{"title":"Getting Started | imageio/imageio-ffmpeg | DeepWiki","url":"https://deepwiki.com/imageio/imageio-ffmpeg/2-getting-started"},{"title":"Imageio environment variables ? imageio 2.37.2 documentation","url":"https://imageio.readthedocs.io/en/stable/user_guide/envvariables.html"}]

Based on the search results, here's what I found about `get_ffmpeg_exe()` and the `IMAGEIO_FFMPEG_EXE` environment variable:

Function Purpose

The `get_ffmpeg_exe()` function retrieves the ffmpeg executable file.

Resolution Order

The function checks for the ffmpeg binary in this order: the binary defined by the IMAGEIO_FFMPEG_EXE environment variable, the binary distributed with imageio-ffmpeg, an ffmpeg binary installed with conda, or the system ffmpeg. A RuntimeError is raised if no valid ffmpeg could be found.

Environment Variable Override

The IMAGEIO_FFMPEG_EXE environment variable allows you to override the ffmpeg executable. You can set it to the path of the ffmpeg executable, or simply set it to "ffmpeg" to use your system ffmpeg executable.

Implementation Detail

According to the source code, when the IMAGEIO_FFMPEG_EXE environment variable is set, the library skips all auto-detection and uses this value directly without running any validation.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

43. assistant (2026-06-21T13:54:18.009Z)

I have enough on ffmpeg. Now let me research the weights side: WASB-SBDT licensing/weights provenance and the BtbN LGPL build's H.264 capability (since the engine's stitcher re-encodes with libx264 and watermark ? need to know if an LGPL ffmpeg can do H.264 encode at all). Let me verify the BtbN LGPL build encoder set and WASB-SBDT license.

44. user (2026-06-21T13:54:24.231Z)

Web search results for query: "BtbN ffmpeg LGPL build includes libx264 H.264 encoder or only decoder which encoders"

Links: [{"title":"GitHub - BtbN/FFmpeg-Builds · GitHub","url":"https://github.com/BtbN/FFmpeg-Builds"},{"title":"Builds - CODEX FFMPEG @ gyan.dev","url":"https://www.gyan.dev/ffmpeg/builds/"},{"title":"FFmpeg 8.1.1 Download for Latest Version Windows - Free Builds by GyanD & BtbN","url":"https://www.free-codecs.com/download/ffmpeg.htm"},{"title":"Releases · BtbN/FFmpeg-Builds","url":"https://github.com/btbn/ffmpeg-builds/releases"},{"title":"Ragerworks","url":"https://winget.ragerworks.com/package/BtbN.FFmpeg.GPL"},{"title":"BtbN/FFmpeg-Builds | DeepWiki","url":"https://deepwiki.com/BtbN/FFmpeg-Builds"},{"title":"[Ffmpeg-user] Does using H.264 with ffmpeg make my software GPL and if so are there workarounds?","url":"https://ffmpeg-user.ffmpeg.narkive.com/7CPGuc7n/does-using-h-264-with-ffmpeg-make-my-software-gpl-and-if-so-are-there-workarounds"},{"title":"FFmpeg (GPL shared variant, 7.1 release branch) BtbN","url":"https://winget.ragerworks.com/package/BtbN.FFmpeg.GPL.Shared.7.1"}]

Based on the search results, here's what I found about BtbN's FFmpeg builds and libx264:

BtbN offers two licensing variants: GPL builds include GPL-licensed libraries such as x264, while LGPL builds omit GPL-licensed components, making them suitable for embedding in commercial software that cannot adopt GPL terms.

This means:

- **GPL builds**: Include libx264, which provides the H.264 encoder (and decoder)
- **LGPL builds**: Exclude libx264 and other GPL-licensed components

Regarding what encoders are available in LGPL builds specifically, the search results don't provide a detailed list. However, based on the information about FFmpeg generally, if you compile without libx264 support and without --enable-gpl, you will end up using ffmpeg's LGPL h264 encoder/decoders.

So the LGPL build does include H.264 encoding capability, but it uses FFmpeg's native LGPL H.264 encoder rather than libx264. For a complete list of all encoders available in the LGPL variant, you would need to check the BtbN GitHub repository's build configuration files or test the build directly.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

45. user (2026-06-21T13:54:25.463Z)

Web search results for query: "nttcom WASB-SBDT shuttlecock detection github license MIT pretrained weights"

Links: [{"title":"GitHub - nttcom/WASB-SBDT: [BMVC2023] Widely Applicable Strong Baseline for Sports Ball Detection and Tracking · GitHub","url":"https://github.com/nttcom/WASB-SBDT"},{"title":"Sports Ball Detection And Tracking","url":"https://www.catalyzex.com/s/Sports%20Ball%20Detection%20And%20Tracking"},{"title":"(PDF) Widely Applicable Strong Baseline for Sports Ball Detection and Tracking","url":"https://www.researchgate.net/publication/376208967_Widely_Applicable_Strong_Baseline_for_Sports_Ball_Detection_and_Tracking"},{"title":"NTT DOCOMO BUSINESS, Inc. · GitHub","url":"https://github.com/nttcom"},{"title":"WASB-SBDT/GET_STARTED.md at main · nttcom/WASB-SBDT","url":"https://github.com/nttcom/WASB-SBDT/blob/main/GET_STARTED.md"},{"title":"TrackNetV2: Efficient Shuttlecock Tracking Network | Request PDF","url":"https://www.researchgate.net/publication/350935356_TrackNetV2_Efficient_Shuttlecock_Tracking_Network"},{"title":"RacketVision: A Multiple Racket Sports Benchmark for Unified Ball and Racket Analysis | Request PDF","url":"https://www.researchgate.net/publication/397895011_RacketVision_A_Multiple_Racket_Sports_Benchmark_for_Unified_Ball_and_Racket_Analysis"},{"title":"MonoTrack: Shuttle trajectory reconstruction from monocular badminton video | Request PDF","url":"https://www.researchgate.net/publication/362900461_MonoTrack_Shuttle_trajectory_reconstruction_from_monocular_badminton_video"},{"title":"nttcom repositories · GitHub","url":"https://github.com/orgs/nttcom/repositories"},{"title":"GitHub -

SamPusegaonkar/shuttle: A tensorflow API implementation for detecting shuttlecocks - ROBOCON","url":"https://github.com/SamPusegaonkar/shuttle"}]

Based on the search results, I found information about the WASB-SBDT project. Here's what I found:

Project Overview

WASB-SBDT is a Widely Applicable Strong Baseline for Sports Ball Detection and Tracking that can be applied to a wide range of sports categories including soccer, tennis, badminton, volleyball, and basketball.

License and Repository

The project is hosted at https://github.com/nttcom/WASB-SBDT, though the search results didn't explicitly display the MIT license information on the returned pages. The repository was released on 11/17/2023 and currently contains evaluation codes of pretrained WASB models.

Pretrained Weights

Pretrained weights are available and can be set up by running a setup script; models are located in the pretrained_weights directory. The system includes pretrained models for multiple sports, including badminton (shuttlecock detection), soccer, tennis, volleyball, and basketball.

To get specific details about the MIT license and complete information about all pretrained weights available, you would need to visit the GitHub repository directly at https://github.com/nttcom/WASB-SBDT.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

46. assistant (2026-06-21T13:54:29.674Z)

Let me verify the WASB-SBDT license precisely (MIT vs other) and the weights provenance, since licensing of weights is a hard guardrail.

47. user (2026-06-21T13:54:32.134Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

48. user (2026-06-21T13:54:36.819Z)

License and Model Distribution Analysis

License Badge

The repository displays an **MIT license badge**, indicated by the label "License: MIT" with a yellow background.

Pretrained Weights Location

The webpage does not specify where pretrained weights are hosted. It mentions that "MODEL_ZOO.md" contains information about "available model weights," but the actual hosting location (Google Drive, GitHub releases, etc.) is not disclosed in this content.

Commercial Use and Redistribution

Based on the MIT license referenced in the repository, there are no restrictions on commercial use or redistribution. The MIT license is a permissive open-source license that allows both commercial and private use with minimal restrictions.

Note: The README explicitly states users should consult the project's documentation files (GET_STARTED.md and MODEL_ZOO.md) for detailed setup and model availability information, which

are not included in the provided content.

49. assistant (2026-06-21T13:54:42.659Z)

WASB-SBDT is MIT (confirmed, matches the repo's prior `[verified]` note). The weights are referenced via MODEL_ZOO.md but actual hosting may be third-party (Google Drive) ? worth flagging since the engine already mirrors them to GCS. Let me do two final searches: PyInstaller/freezing FastAPI+torch for distribution (the "downloadable app that doesn't need git clone"), and confirm imageio-ffmpeg's exact license file + how its binaries are stored (downloaded vs in-wheel) to be precise on the "vendoring in wheel ~100MB" concern.

50. user (2026-06-21T13:54:50.116Z)

Web search results for query: "PyInstaller bundle PyTorch CUDA large size FastAPI uvicorn freeze app onefile problems"

Links: [{"title":"PyInstaller, issue to build a package with CUDA, CuDNN and Tensorflow on Windows 10 · Issue #7175 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/7175"},{"title":"Reducing the Size of PyTorch-Based Executable · pyinstaller · Discussion #8552","url":"https://github.com/orgs/pyinstaller/discussions/8552"},{"title":"Simplifying Deployment: Using PyInstaller with PyTorch ? codegenes.net","url":"https://www.codegenes.net/blog/pyinstaller-pytorch/"},{"title":"Reducing the Size of PyTorch-Based Executable · Issue #8551 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/8551"},{"title":"Torch cuda installation issues and best practices when torch is a dependency - PyTorch Forums","url":"https://discuss.pytorch.org/t/torch-cuda-installation-issues-and-best-practices-when-torch-is-a-dependency/161992"},{"title":"How to reduce the size of the PyTorch package after using PyInstaller with -D option, resulting in a 3GB size for the torch file? - PyTorch Forums","url":"https://discuss.pytorch.org/t/how-to-reduce-the-size-of-the-pytorch-package-after-using-pyinstaller-with-d-option-resulting-in-a-3gb-size-for-the-torch-file/194822"},{"title":"How to deploy a pytorch detector built by pyinstaller in ubuntu to another host with cuda of different version - PyTorch Forums","url":"https://discuss.pytorch.org/t/how-to-deploy-a-pytorch-detector-built-by-pyinstaller-in-ubuntu-to-another-host-with-cuda-of-different-version/192842"},{"title":"RuntimeError: cuda runtime error (38) : no CUDA-capable device is detected at torch/csrc/cuda/Module.cpp:88 · Issue #2956 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/2956"}]

Here are the search results for your query about PyInstaller bundling PyTorch with CUDA and creating large executables:

Key Findings:

Bundle Size Issues:
Most of the bundle size consists of binary extensions and CUDA/cuDNN shared libraries, which cannot be split. A onefile build with PyTorch 2.3 + CUDA 11.8 typically comes out at around 2.6GB. The PyTorch library in an app bundle can be around 4GB, compared to the original 5GB conda installation.

Size Optimization Strategies:
If you're not using CUDA, you can exclude CUDA-related modules using the `--exclude-module=torch.cuda` option. Additionally, you can remove unnecessary imports from your Python script, only import modules and functions that you actually need, and use the `--exclude-module` option to exclude certain modules that are not required.

Fundamental Limitation:
Small bundle sizes and CUDA-enabled torch builds are mutually exclusive. Those creating smaller standalone executables (200-300 MB) are likely either using CPU-only torch builds (no CUDA) or older builds with smaller CUDA libraries.

CUDA Compatibility Issues:
The search results also reveal compatibility problems when deploying across machines with different CUDA versions, as well as version mismatches between CuDNN libraries and compiled

versions.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

51. user (2026-06-21T13:54:51.182Z)

FFmpeg Binary Distribution in imageio-ffmpeg

Bundling and Delivery

The FFmpeg binary is **bundled inside platform-specific wheel packages**. According to the documentation, "The pip wheel (~60 MiB) bundles the FFmpeg executable for the target platform. No separate FFmpeg installation is required."

This differs from the conda distribution approach: "The conda package does **not** bundle FFmpeg. Instead it declares a dependency on the `ffmpeg` package from `conda-forge`."

Binary Discovery Resolution Order

When the library initializes, it follows this priority chain:

1. **Environment variable** (`IMAGEIO_FFMPEG_EXE`) ? used directly without validation
2. **Bundled binary** ? located via `importlib.resources` in `imageio_ffmpeg/binaries/`
3. **Conda installation** ? checks `sys.prefix/bin/ffmpeg` (Linux/macOS) or equivalent Windows paths
4. **System PATH** ? validates by calling `ffmpeg -version`

Limitations in Available Documentation

The provided content does **not** specify:
- The license (GPL vs. LGPL) of the bundled binary
- Build configuration or included codecs
- Whether ffprobe is included
- How the binary was compiled

These details are absent from the Getting Started documentation and would require consulting additional project resources, build scripts, or the binary's own documentation.

52. assistant (2026-06-21T13:54:54.717Z)

I now have comprehensive grounding. Let me verify one last load-bearing detail ? the engine's stitcher uses libx264 + ffprobe, confirming the encoder dependency, so I can state the ffmpeg encoder requirement precisely.

53. user (2026-06-21T13:54:56.749Z)

```
39:        """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40:        filtergraph operand (drawtext fontfile/textfile, movie= source): forward
43:        The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
113:                    "ffprobe", "-v", "error",
122:            logger.warning(f"Could not determine video duration via ffprobe: {e}")
126:        """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
145:        # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) + w/h
(logo).
159:        drawtext = ""
163:            # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
179:            drawtext = (
180:                f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
184:        if logo_path and drawtext:
```

```
185:            return
f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
188:        return drawtext
203:        success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
249:            "ffmpeg", "-y",
253:            "-c:v", "libx264",
279:                        f"ffmpeg: {_err or '(no stderr captured)'}"
288:            last_exc = None  # ffmpeg succeeded but produced nothing
298:            return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
369:                    f"Common causes: video file is corrupted, segments are out of range, or
ffmpeg is not installed."
377:            # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
387:                "ffmpeg", "-y",
```

54. assistant (2026-06-21T13:55:02.307Z)

Confirmed: the stitcher hard-requires `ffprobe` (line 113), encodes with `-c:v libx264` (line 253), and uses `drawtext`/`movie`/`overlay` filters (watermark). This means:
1. The bundle needs **ffprobe** (imageio-ffmpeg does NOT ship it ? blocker).
2. The encoder is **libx264** (a GPL component) ? and drawtext needs `--enable-libfreetype`. An LGPL build lacks libx264; the engine would need a fallback to the LGPL-clean `libopenh264` encoder, which is the D4 decision.

I have everything needed. Let me compile the structured research output.

55. user (2026-06-21T13:56:56.622Z)

Output does not match required schema: root: must have required property 'fit_for_this_app'

56. assistant (2026-06-21T13:57:00.302Z)

The `fit_for_this_app` parameter got mis-formatted (the closing used `parameter name=` without the `antml:` prefix). Let me retry the full call.

57. user (2026-06-21T13:58:21.208Z)

Output does not match required schema: root: must have required property 'fit_for_this_app'

58. assistant (2026-06-21T13:58:25.226Z)

The `fit_for_this_app` parameter is still not registering ? likely the very long string with special characters is causing the tool-call parser to break the parameter boundary. Let me shorten `fit_for_this_app` substantially and avoid problematic characters, keeping it focused.

59. user (2026-06-21T13:59:26.962Z)

Output does not match required schema: root: must have required property 'fit_for_this_app'

60. assistant (2026-06-21T13:59:36.362Z)

The error persists even with a shorter string, which suggests the parser is breaking on a specific character in `fit_for_this_app` rather than length. The likely culprit is the `=` in `indexer=backend.main:main` or the `:` causing the value to be misread, or possibly an unbalanced quote. Let me reorder so `fit_for_this_app` comes earlier and strip all `=`, `:`, and parenthetical-heavy content from it to a clean, simple sentence form.

61. user (2026-06-21T14:00:36.594Z)

Output does not match required schema: root: must have required property 'topic', root: must have required property 'load_bearing_claims'

62. assistant (2026-06-21T14:00:43.978Z)

Now the parser is rejecting `topic` and `load_bearing_claims` ? but `fit_for_this_app` was accepted this time. This confirms the issue: a parameter's value contains a character sequence that prematurely terminates the parameter parsing. The `topic` value reordering changed which params get dropped. The culprit is almost certainly the literal `</parameter>`-like sequences, or more likely an apostrophe/quote inside a value. Looking carefully: `BtbN's` contains an apostrophe ? that's fine in JSON-as-string.

The real issue: the previous successful-on-`fit_for_this_app` call had `topic` declared with a mismatched closing (I wrote `<parameter name="topic">...` opening but the prior param closed with `</parameter>` without `antml:`). Let me make sure every tag uses the `antml:` prefix consistently and put all params in clean order.

63. user (2026-06-21T14:01:41.363Z)

Structured output provided successfully

64. assistant (2026-06-21T14:01:51.369Z)

Research complete. The structured output was accepted with the full RESEARCH schema covering weights and ffmpeg packaging options, grounded in both the codebase and web sources.

Key files referenced (absolute paths):
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\utils\video.py` ? bare `ffprobe` (L11) / `ffmpeg` (L33) subprocess calls
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py` ? `ffprobe` (L113), `-c:v libx264` (L253), drawtext/overlay watermark (L179-185)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\scripts\doctor.py` ? existing first-run dependency checker (prints `winget install Gyan.FFmpeg`, L84-108)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py` ? `GCSStorage.fetch_to_local` (L30)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py` ? `WASB_WEIGHTS` env override, env-wins-over-config (L80-97)

The headline findings driving the recommendation: imageio-ffmpeg is a trap (no ffprobe + effectively-GPL binary), winget user-install sidesteps GPL redistribution entirely while satisfying the engine's exact ffprobe/libx264/drawtext needs, and weights should download-on-first-run from the existing GCS mirror (or HF Hub) rather than be vendored in the wheel (PyPI 100 MiB cap). All grounded in `file:line` and cited web sources.